SpiralTrain
developer courses



Python Programming

General Course Remarks

- Course times
- Drinks
- Meals
- Toilets
- Smoking
- Mobile Phones
- Manual
- Software

Everything should be made as simple as possible, but not simpler

Albert Einstein

Ja, was ich hier geschrieben habe, macht im Einzelnen überhaupt nicht den Anspruch auf Neuheit; und darum gebe ich auch keine Quellen an, weil es mir gleichgültig ist, ob das was ich gedacht habe, vor mir schon ein anderer gedacht hat.

Ludwig Wittgenstein

SpiralTrain developer courses

- ▶ **Python Intro**
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

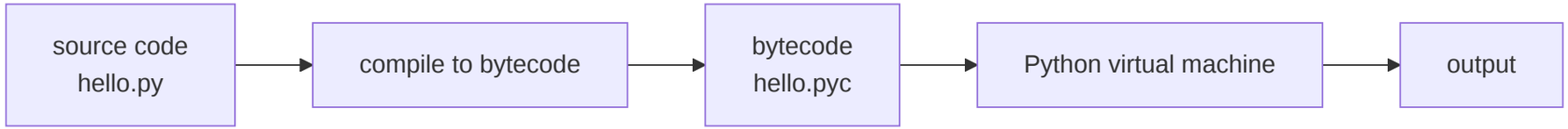
- Type Hinting
- Python Libraries

- What is Python?
- Python Features
- History of Python
- Modern Python Tooling
- Getting Started
- Setting up PATH
- Creating a Project
- Adding Dependencies
- Running Python
- Python in Interactive Mode
- Python in Script Mode
- Linting with ruff
- Editor Setup
- Python Identifiers
- Python Reserved Words
- Comments in Python
- Lines and Indentation
- Multi Line Statements
- Quotes in Python

SpiralTrain developer courses

What is Python?

- **Python is a general purpose high-level scripting language :**
  - Supports many applications from text processing to WWW browsers and games
- **Python is Beginner's Language :**
  - Great language for the beginner programmers
- **Python is interpreted :**
  - Processed at runtime by the interpreter, with no separate compile step to run



- **Python is interactive :**
  - Prompt allows you to interact with the interpreter directly to write your programs
- **Python is Object Oriented :**
  - Supports Object-Oriented style programming that encapsulates code within objects
- **Python is easy to learn and easy to read :**
  - Python has relatively few keywords, simple structure, and a clearly defined syntax
  - Python code is much more clearly defined and visible to the eye

SpiralTrain developer courses

Python Features

- **Broad standard library :**
  - Files, paths, dates, JSON, CSV, regular expressions, SQLite and testing out of the box
- **Portable :**
  - The same code runs on Linux, Windows and macOS
- **Large ecosystem :**
  - Hundreds of thousands of packages on PyPI, installed with `uv add`
- **Used across many fields :**
  - Automation and scripting, web back ends, data analysis, machine learning, testing
- **Extendable :**
  - Performance-critical parts can be written in C or Rust and called from Python
- **Automatic memory management :**
  - Objects are freed by reference counting and a garbage collector, not by hand

SpiralTrain developer courses

History of Python

- **Python was developed by Guido van Rossum around 1990 at :**
  - National Research Institute for Mathematics and Computer Science in Netherlands
- **Python is derived from many other languages :**
  - Including Modula-3, C, C++, Smalltalk, Unix shell and other scripting languages
- **Python is usable as :**
  - Scripting language or compiled to byte-code for building large applications
- **Python is copyrighted and the source is open :**
  - Released under the PSF License, a permissive licence that allows commercial use
- **Python is developed in the open by a community of core developers :**
  - The Python Software Foundation holds the rights, the Steering Council decides
- **Python supports :**
  - Functional and structured programming methods as well as OOP
  - Very high-level dynamic data types and dynamic type checking

SpiralTrain developer courses

Modern Python Tooling

- **Python needs an interpreter, an environment and a set of packages :**
  - Historically each of these was installed and managed by a separate tool
  - The result was hard to reproduce on a second machine
- **Two tools now cover almost all of it :**
  - `uv` installs Python, creates the environment and resolves the packages
  - `ruff` checks style and catches a large class of mistakes before you run
- **Both are single binaries with no Python of their own to bootstrap :**

Older tool	Replaced by
`python.org` installer, `pyenv`	`uv python install`
`venv`, `virtualenv`	`uv` creates `.venv` for you
`pip`, `pip-tools`, `requirements.txt`	`uv add` and `uv.lock`
`flake8`, `isort`, `black`, `pylint`	`ruff check` and `ruff format`

SpiralTrain developer courses

Getting Started

- Install `uv` once, and let it install Python for you :

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

- On macOS or Linux use the shell installer :

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- Close and reopen the terminal, then fetch the interpreter :

```
uv python install 3.14
```

- This needs no administrator rights :
  - Any Python already on the machine is left alone, and versions live side by side
- The traditional route is the installer from `https://www.python.org/` :
  - Tick `Add python.exe to PATH` on the first page of the Windows wizard
  - Python 2 reached its end of life in 2020, so Python 3 is the only choice
- Documentation lives at `https://docs.python.org/3/`

SpiralTrain developer courses

Setting up PATH

- **Only needed when Python was installed by hand :**
  - `uv` puts the interpreter where `uv run` can find it, so it needs none of this
- **PATH is an environment variable :**
  - Provides a search path that lists directories that OS searches for executables
  - Named PATH in Unix or Path in Windows, and Unix is case-sensitive
- **Setting PATH in Linux :**

```
export PATH="$PATH:/usr/local/bin/python"
```

- **Setting PATH in Windows :**

```
PATH=%PATH%;C:\Python
```

- **Or use the environment variables tab under System Properties**
- **PYTHONPATH is a separate variable :**
  - It tells the interpreter where to find modules you import, not where Python is

Creating a Project

- **A project is a directory with a `pyproject.toml` in it :**

```
uv init course-work
cd course-work
```

- **`uv init` writes three things :**
  - `pyproject.toml` , the file that names the project and its dependencies
  - `.python-version` , the interpreter this project is pinned to
  - `main.py` , a placeholder to delete or edit

```
course-work/
├─ pyproject.toml
├─ .python-version
├─ main.py
└─ .venv/              created on the first uv run or uv add
```

- **Run code through `uv run` , which uses the project's own interpreter :**

```
uv run python main.py
```

SpiralTrain developer courses

Adding Dependencies

- `uv add` installs a package and records it in `pyproject.toml` :

```
uv add requests
uv add "requests>=2.28,<3"
uv add --dev pytest
```

- Two files describe the dependencies, and both belong in version control :
  - `pyproject.toml` states what the project asks for
  - `uv.lock` states the exact versions that were resolved
- Anyone can then reproduce the environment exactly :

```
uv sync          # install exactly what uv.lock says
uv remove requests
```

- `--dev` marks a tool used while developing, not by the running program :
  - `ruff` is such a tool; so is the test runner `pytest`, which the libraries appendix covers
- There is no environment to activate :
  - `uv run` puts the project's `.venv` in front for the length of one command

demo01
uv_project

SpiralTrain developer courses

Running Python

- **Inside a project, `uv run python` starts the project's own interpreter :**
  - A plain `python` works too, where a Python is on the PATH
- **Interactive Interpreter :**
  - Start the interactive interpreter by entering `uv run python` with no file
  - The interpreter answers with its `>>>` prompt, ready for a statement
  - `python -h` lists the command line options
- **Script from Command Line :**
  - A Python script is executed by passing its file name to the interpreter

```
uv run python script.py
```

- **Integrated Development Environment :**
  - Run and debug Python from an editor, which is how most work is done
  - This course uses VS Code, set up two slides on
- **Jupyter Notebook on a web server :**
  - Cell by cell execution, common in data science work

SpiralTrain developer courses

Python in Interactive Mode

- **Start with Hello World Python program in interactive mode :**
  - Invoke the interpreter by typing `uv run python` in a terminal

```
C:\course-work> uv run python
Python 3.14.0 (main, Oct  7 2025, 10:12:33) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, Python!")
Hello, Python!
>>>
```

- **Type the statement right of the `>>>` prompt and press Enter :**
  - The interpreter evaluates it and prints the result immediately
- **Separate multiple statements with a semicolon ;**

SpiralTrain developer courses

Python in Script Mode

- **When the interpreter is invoked with a script parameter :**
  - The script begins execution and continues until it is finished
  - When the script is finished, the interpreter is no longer active
- **All Python files will have extension `.py` :**
  - Put the following source code in a `hello.py` file

```
print("Hello, Python!")
```

- **Run the script as follows :**

```
uv run python hello.py
```

- **This will produce following result :**

```
Hello, Python!
```

- **`print()` default prints a new line**

demo02
hellopython

SpiralTrain developer courses

Linting with ruff

- **A linter reads code without running it and reports what looks wrong :**
  - Unused imports and variables, shadowed built-ins, unreachable code
  - Style that departs from the conventions the rest of Python follows
- **Two commands cover the daily use :**

```
uv run ruff check .          # report problems
uv run ruff check . --fix    # fix the ones that are safe to fix
uv run ruff format .         # lay the code out consistently
```

- **Rules are configured in `ruff.toml`, which ships with this course :**
  - It selects the rule set the course material is written against
  - Without it the editor would flag patterns the course uses on purpose

demo03
ruff_findings

SpiralTrain developer courses

Editor Setup

- **The course demonstrates Visual Studio Code :**
  - Any editor works, but the slides assume this one
- **Install two extensions from the Extensions panel :**
  - `Python` , published by Microsoft
  - `Ruff` , published by Astral
- **With both installed the editor will :**
  - Underline the problems `ruff check` would report, as you type
  - Offer the same fixes, and format on save if you ask it to
  - Resolve imports against the project's `.venv` rather than a global Python
- **Point the editor at the project interpreter :**
  - Command Palette, `Python: Select Interpreter` , choose the one in `.venv`

SpiralTrain developer courses

Python Identifiers

- **Identifiers are the names of variables, functions, classes and modules :**
  - Start with a letter or an underscore, followed by letters, digits and underscores
  - Punctuation such as `@`, `$` and `%` is not allowed in a name
- **Python is case sensitive :**
  - `total` and `Total` are two different names
- **The naming conventions of PEP 8, the Python style guide :**
  - `snake_case` for variables, functions and modules
  - `CapWords` for classes, `UPPER_CASE` for values meant as constants
- **Underscores carry meaning :**
  - A leading underscore, as in `_cache`, marks a name as internal to its module
  - Names such as `__init__` with two underscores on both sides are defined by Python

Python Reserved Words

- **Keywords may not be used as a variable, function or other name :**

```
False      None       True       and        as         assert     async
await      break      class      continue   def        del        elif
else       except     finally    for        from       global     if
import     in         is         lambda     nonlocal   not        or
pass       raise      return     try        while      with       yield
```

- **`match`, `case`, `type` and `_` are soft keywords :**
  - Reserved only inside the statement they belong to, and usable as names elsewhere
- **The interpreter can list them : `import keyword` and print `keyword.kwlist`**

SpiralTrain developer courses

Comments in Python

- **Comments starts with :**
  - A hash sign (#) that is not inside a string literal
  - All characters after the # and up to the physical line end are part of the comment

```
# First comment
print("Hello, Python!")  # second comment
```

- **Will produce following result :**

```
Hello, Python!
```

- **Comment may be on same line after statement or expression :**

```
name = "Einstein"  # This is again comment
```

- **Comment multiple lines as follows :**

```
# This is a comment.
# This is a comment, too.
# This is a comment, too.
# I said that already.
```

SpiralTrain developer courses

Lines and Indentation

- **Blocks of code are denoted by line indentation :**
  - No braces for blocks of code for class and function definitions or flow control
- **Number of spaces in the indentation is variable :**
  - All statements within the block must be indented the same amount

```
if True:  # Both blocks in first example are fine
    print("True")
else:
  print("False")

if True:  # Second line in second block will generate an error
    print("Answer")
    print("True")
else:
    print("Answer")
  print("False")
```

- `print()` **without new line is written as** `print("hello python", end="")`
- **Semicolon ; allows multiple statements on single line**

demo04
lines_and_identation

SpiralTrain developer courses

Multi Line Statements

- Statements in Python typically end with a newline
- Python allows the use of the line continuation character `(\)` :
  - Denotes that the line should continue

```
total = item_one + \
        item_two + \
        item_three
```

- Statements contained within the `[], {},` or `()` brackets :
  - No need to use the line continuation character

```
days = ["Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday"]
```

- On a single line :
  - Semicolon (;) allows multiple statements on single line
  - Neither statement should start a new code block

demo05
multilinestatements

SpiralTrain developer courses

Quotes in Python

- **Python uses quotes to denote string literals :**
  - single (’), double (”) and triple (’’’ or ”””)
  - Same type of quote should start and end the string
  - Triple quotes can be used to span the string across multiple lines

```
print('hello' + " " + '''world''')
word = 'word'
sentence = "This is a sentence."
paragraph = """This is a paragraph. It is made up of
               multiple lines and sentences."""
```

- **Single quote in a single quoted string :**
  - Displayed with an escape character \

```
print('That\'s a brave new world')
```

demo06
quotesandescapes

SpiralTrain developer courses

Summary : Python Intro

- **Python is a general purpose high-level scripting language :**
  - Interpreted, interactive and object oriented, developed by Guido van Rossum
- **`uv` manages the interpreter, the environment and the packages :**
  - `uv python install`, `uv init`, `uv add`, `uv run`
  - `pyproject.toml` states what is wanted, `uv.lock` pins what was resolved
- **`ruff` reports style and correctness problems without running the code**
- **Python can be started in three different ways :**
  - Interactive interpreter, script from the command line, from inside an IDE
- **Python is case sensitive, and names follow PEP 8 :**
  - `snake_case` for variables and functions, `CapWords` for classes
- **Python uses line indentation to denote blocks of code :**
  - No braces for class and function definitions or for flow control
- **The line continuation character () denotes that a line should continue**
- **Quotes denote string literals, single, double or triple**

exercise01
Python Intro

SpiralTrain developer courses

- Python Intro
- ▶ **Variables and Data Types**
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

- Type Hinting
- Python Libraries

- Variables
- Everything is an Object
- Multiple Assignment
- Strong Typing
- Annotating Variables
- Python Numbers
- Type Conversions
- Conversion Functions
- Built-in Number Functions
- Python Operators
- Arithmetic and Assignment Operators
- Comparison Operators
- Logical, Membership, Identity Operators
- Python Operators Precedence
- Python Strings
- String Operators
- String Operations
- Escape Characters
- Raw and Unicode Strings
- f-Strings
- Format Specifications
- Triple Quotes
- Built-in String Methods

SpiralTrain developer courses

Variables

- **Variables are names bound to objects :**
  - Assignment binds a name to the object produced by an expression
  - Several names can refer to the same object
- **Objects have types, such as `int`, `float` and `str` :**
  - `type(x)` returns the type of the object currently bound to `x`
  - Rebinding `x` can make it refer to an object of a different type
- **`id(x)` identifies the object while it exists**
- **The `=` operator binds the name on its left to the object on its right :**
  - A name does not need a separate declaration before assignment

```
counter = 100  # An integer assignment
miles = 1000.0  # A floating point
name = "John"  # A string
```

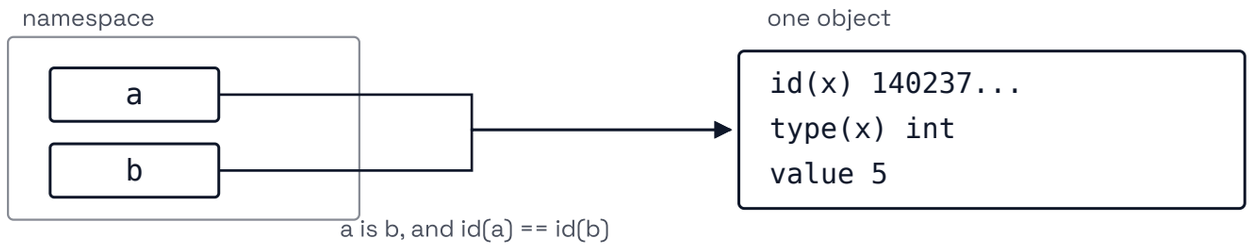
- **Python language does not have constants**

demo01
variables

SpiralTrain developer courses

Everything is an Object

- **Each object `x` has :**
  - Unique identifier, an integer, returned by `id(x);`
  - Type returned by `type(x)`
  - Value may or may not be changeable :
    - Value can itself consist of one or more objects
  - Zero or more methods provided by type object to get and/or change content
  - Zero or more names
- **Objects have individuality :**
  - Multiple names in multiple scopes can be bound to the same object
  - This is known as aliasing
- **Variable names are not properties of an object :**
  - A namespace maps each name to the object it refers to



demo02
typeandid

SpiralTrain developer courses

Multiple Assignment

- **Simultaneous assignment of values :**

```
a = b = c = 1
```

- **All three names are bound to the same integer object**
- **Reassigning one name changes only that binding :**
  - The other names still refer to the original object
- **Integers are immutable :**
  - Rebinding a name does not change the integer it previously named
  - The new binding may refer to an already existing object
- **Assignment of multiple objects to multiple variables :**

```
a, b, c = 1, 2, "john"
```

- **Two integer objects with values 1 and 2 :**
  - Assigned to variables a and b
- **One string object with the value ”john” :**
  - Assigned to the variable c

demo03
multipleassignment

SpiralTrain developer courses

Strong Typing

- **Data stored in memory can be of many types :**
  - Persons age is stored as a numeric value
  - Persons address is stored as alphanumeric characters
- **A type (or class) of a variable determines :**
  - Operations possible on the variable
  - Storage method for the variable
- **Python is a strongly typed language :**
  - An operation the type does not allow raises rather than coercing silently

```
1 + 1      ->  2             int plus int
"1" + "1"  ->  '11'          str plus str, concatenation
"1" + 1    ->  TypeError     no implicit conversion between the two
```

- **Important Python data types :**
  - Numbers, String, List, Tuple, Set and Dictionary

demo04
strongtyping

Annotating Variables

- **A variable can carry a type hint saying what it is meant to hold :**

```
age: int = 25
height: float = 1.87
name: str = "Alice"
is_student: bool = True
```

- **The hint changes nothing at runtime :**
  - Python still stores whatever it is given, and still checks types only when an operation is attempted
  - The annotation records the intent, so that a reader and a tool can see it
- **It is worth writing where the type is not obvious from the value :**
  - `total: float = 0` says that the running total will not stay an integer
  - An editor can then complete and check against the annotation
- **Annotations appear throughout the rest of this course :**
  - Collections in the data structures module, functions in the functions module
  - The appendix gathers them up and adds a checker

SpiralTrain developer courses

Python Numbers

- **Assignment binds names to number objects :**

```
var1 = 1
var2 = 10
```

- **Numbers are immutable data types :**
  - Reassignment binds a name to another number; it does not modify the old one

```
var1 = 11
var1 = 2
var2 = 1
```

- **Remove one or more name bindings with `del`; other references keep the objects alive :**

```
del var
del var_a, var_b
```

SpiralTrain developer courses

Type Conversions

- **Sometimes number must be explicitly converted :**
  - Coerced from one type to another to satisfy requirements of function parameter
- **Built-in functions perform conversion between data types :**
  - `int(x [,base]) :`
    - Converts x to a plain integer, base specifies the base if x is a string
  - `float(x) :`
    - Converts x to a floating-point number
  - `bool([x])` :
    - Convert value to Boolean using standard truth testing procedure
- **All integers, positive and negative behave as `True` :**
  - Only 0 behaves as `False`
- **All non empty strings behave as `True` :**
  - Only empty string `""` behave as `False` and `None` as well
- **Boolean type is a subtype of the integer type :**
  - Boolean values behave like values 0 and 1
  - When converted to a string in string context `"False"` or `"True"` are returned

demo05
numbertypeconversion

SpiralTrain developer courses

Conversion Functions

- **Functions to convert between built-in types :**

Function	Description
`str(x)`	Converts object x to a string representation
`repr(x)`	Converts object x to an expression string
`eval(expression)`	Evaluates a Python expression and returns result
`chr(x)`	Converts an integer Unicode codepoint to a character
`ord(x)`	Converts character to integer representing Unicode code point
`hex(x)`	Converts an integer to a hexadecimal string
`oct(x)`	Converts an integer to an octal string

```
y2 = repr("abc")  # y2 is "'abc'"
y3 = str("abc")   # y3 is 'abc'
i = 65
print(chr(i))    # A
print(hex(i))    # 0x41
print(ord("A"))  # 65
```

demo06
conversionfunctions

SpiralTrain developer courses

Built-in Number Functions

Function	Description
`abs(x)`	Absolute value of x: the (positive) distance between x and zero
`ceil(x)`	The ceiling of x: the smallest integer greater than or equal to x
`exp(x)`	The exponential of x: e**x
`fabs(x)`	The absolute value of x, is part of math module
`floor(x)`	The floor of x: the largest integer not greater than x
`log(x)`	The natural logarithm of x, for x > 0
`log10(x)`	The base-10 logarithm of x, for x > 0
`pow(x, y)`	The value of x**y
`sqrt(x)`	The square root of x, for x >= 0
`max(x, y, ..)` / `min(x, y, ..)`	The maximum or minimum of a list of numbers

- **For some of these functions the `math` module must be imported**

demo07
numberfunctions

SpiralTrain developer courses

Python Operators

- An operator combines one or more operands into a new value :

```
print(7 / 2)  # 3.5, true division always gives a float
print(7 // 2)  # 3, floor division drops the fraction
print(7 % 2)  # 1, the remainder
print(2**10)  # 1024, exponentiation
```

- Python groups its operators as follows :
  - Arithmetic and assignment operators, for numbers and for strings
  - Comparison operators, which give `True` or `False`
  - Logical, membership and identity operators
- Bitwise operators `& | ^ ~ << >>` work on the bits of integers, rarely needed
- There is no `++` or `--` : write `count += 1` instead

demo08
pythonoperators

Arithmetic and Assignment Operators

Operator	Arithmetic Operator Description
`+` / `-`	Adds or subtracts the values on either side of the operator
`*`	Multiplies the values on either side of the operator
`/`	Divides left operand by right operand, the result is always a float
`//`	Floor division, the quotient rounded down to a whole number
`%`	Modulo, the remainder of the division
`**`	Exponentiation, the left operand raised to the power of the right

Operator	Assignment Operator Description
`=`	Binds the name on the left to the value on the right
`+=` / `-=`	Adds or subtracts the right operand, and assigns the result to the left
`*=` / `/=`	Multiplies or divides by the right operand, and assigns the result
`//=` / `%=` / `**=`	The same for floor division, modulo and exponentiation

```
count = 10
count += 1
count *= 2
print(count)  # 22
```

Comparison Operators

Operator	Comparison Operator Description
`==` / `≠`	True if the two values are equal, or not equal
`>` / `<`	True if the left value is greater, or less, than the right
`⩾` / `⩽`	True if the left value is greater or equal, or less or equal

- **Comparisons chain, so a range check reads as it would on paper :**

```
i = 10
j = 11
print(i > j)  # False
print(i != j)  # True
print(0 < i < 20)  # True, comparisons chain
print("apple" < "banana")  # True, strings compare letter by letter
```

- **`==` compares values; whether two names are one object is `is`, on the next slide**

Logical, Membership, Identity Operators

SpiralTrain developer courses

Operator	Description
`and` / `or`	True if both operands are true, or if at least one is

- `and` and `or` stop as soon as the answer is known, and return an operand :

```
name = ""
print(name or "anonymous")  # anonymous, `or` returns the first truthy operand
print(name and "anonymous")  # an empty line, `and` returns the first falsy operand
```

- `x if c else y` gives `x` when `c` is true, and `y` when it is false :

```
age = 17
print("adult" if age >= 18 else "minor")  # minor
```

- `is` asks whether two names refer to one object, `==` whether values are equal :

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a == c)  # True, equal values
print(a is c)  # False, two different lists
print(a is b)  # True, two names for one list
```

demo09
booleanoperators

SpiralTrain developer courses

Python Operators Precedence

Operator	Description, from highest to lowest
`**`	Exponentiation, which groups from the right
`+x` `-x` `~x`	Unary plus, minus and bitwise not
`*` `/` `//` `%`	Multiplication, division, floor division and modulo
`+` `-`	Addition and subtraction
`<<` `>>` `&` `^` `|`	Bitwise shifts, and, exclusive or, or
`==` `!=` `<` `<=` `>` `>=` `in` `not in` `is` `is not`	Comparisons, membership and identity
`not`	Logical not
`and`	Logical and
`or`	Logical or

```
print(2 + 3 * 4)  # 14
print((2 + 3) * 4)  # 20
print(2**3**2)  # 512, ** groups from the right
```

- **When in doubt, add parentheses : they cost nothing and help the reader**

SpiralTrain developer courses

Python Strings

- **Strings are sequences of characters between quotation marks :**
  - Pairs of single `'` quotes, of double `"` quotes and of triple (''' or """) quotes supported
  - There is no distinction between single and double quoted strings
- **Creating strings is as simple as assigning a value to a variable :**

```
var1 = "Hello World!"
var2 = "Python Programming"
```

- **Python does not support a character type :**
  - Characters are treated as strings of length one, thus also considered a substring
  - To access substrings, use square brackets for slicing along with the index

```
print("var2[1:5]:", var2[1:5])
```

- **Indexes start at 0 in beginning of the string, then upwards and to -1 at the end**
- **Strings are immutable data types like Numbers :**
  - Changing the value of a string data type results in a newly allocated object

String Operators

Operator	Description
`+`	Concatenation. Adds values on either side of the operator
`*`	Repetition. Creates string by concatenating multiple copies of same string
`[]`	Slice. Gives the character from the given index
`[:]`	Range Slice. Gives the characters from the given range
`in`	Membership. Returns true if a character exists in the given string
`not in`	Membership. Returns true if a character does not exist in the given string
`r/R`	Raw String. Suppresses actual meaning of escape characters

SpiralTrain developer courses

String Operations

- **Examples :**

```
print(str1)  # Prints complete string
print(str1[0])  # Prints first character of the string
print(str1[-1])  # Prints last character of the string
print(str1[2:5])  # Prints characters starting from 3rd through 5th
print(str1 * 2)  # Prints string two times
print(str1 + "TEST")  # Prints concatenated string
print(str1[::-1])  # Prints string backwards
```

- **String updated by assigning part of it to new string :**
  - New value can be related to its previous value or to a completely different string

```
var1 = "Hello World!"
var2 = var1[:6] + "Python"
print("Updated String :-", var2)
```

- **Result :**

```
Updated String :- Hello Python
```

- **New string var2 will have different id**

demo10
stringoperations

SpiralTrain developer courses

Escape Characters

- **Escape or non-printable characters :**
  - Characters that can be represented with backslash notation

Backslash Notation	Hexadecimal Character	Description
`\`	0x5c	Backslash
`\'` / `\"`	0x27 / 0x22	Single or double quote
`\n`	0x0a	Newline
`\r`	0x0d	Carriage return
`\t`	0x09	Tab
`\a`	0x07	Bell or alert
`\b`	0x08	Backspace
`\ooo`		Character with octal value ooo, digits 0 to 7
`\xhh`		Character with hexadecimal value hh
`\uxxxx`		Unicode character with 16-bit hexadecimal value xxxx

demo11
escapecharacters

SpiralTrain developer courses

Raw and Unicode Strings

- **Raw String don't treat backslash as a special character :**
  - Every character in a raw string stays the way it was
- **In a normal String the backslash can be escaped :**

```
print("C:\\nowhere")
```

- **Result :** `C:\nowhere`
- **A raw string is made with the** `r` **in front of the String :**

```
print(r"C:\nowhere")
```

- **Result :** `C:\nowhere`
- **Python strings are Unicode :**
  - A string is a sequence of code points, so `len("Ä…Ä‡")` is 2
  - Text in almost every script in the world can be stored and processed
- `unicodedata` **module** `:`
  - Provides access to Unicode Character Database (UCD)
  - UCD contains character properties for all Unicode characters

SpiralTrain developer courses

f-Strings

- **An f-string is a string literal with an `f` in front of the opening quote :**
  - Each `{expression}` inside it is evaluated, and its value takes its place
  - Any expression works, including arithmetic and method calls

```
name = "Albert"
age = 55
print(f"My name is {name} and my age is {age}")
print(f"{name.upper()} will be {age + 1} next year")
```

```
My name is Albert and my age is 55
ALBERT will be 56 next year
```

- **Two markers after the expression change what is shown :**
  - `!r` shows the value as `repr()` does, a string with its quotes
  - `=` shows the expression as well as its value, which helps when debugging

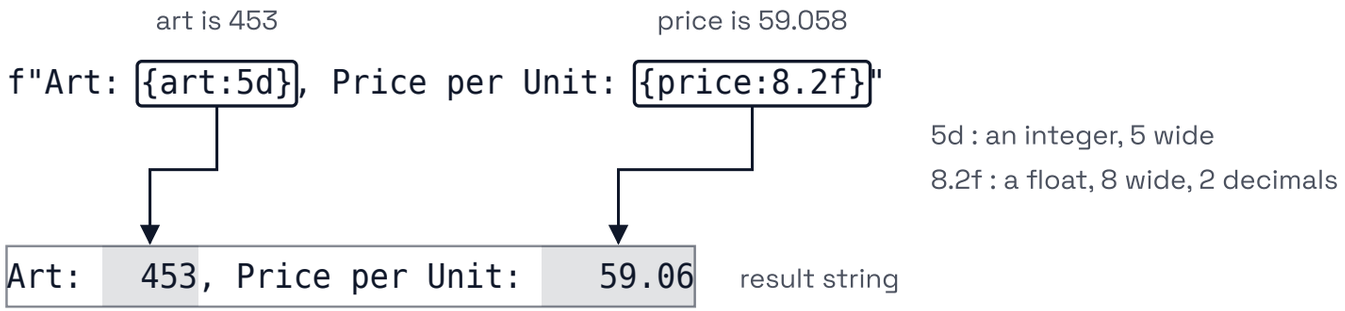
```
print(f"{name!r}")  # 'Albert', the repr() with its quotes
print(f"{age = }")  # age = 55, the expression and its value
```

demo12
f-strings

SpiralTrain developer courses

Format Specifications

- A colon after the expression starts a format specification :



- The specification sets the width, the alignment and the decimals :

```
print(f"[{'Curie':<10}]")  # [Curie     ]
print(f"[{'Curie':^10}]")  # [  Curie   ]
print(f"[{'Curie':>10}]")  # [     Curie]
print(f"{1234567.891:,.2f}")  # 1,234,567.89
print(f"{0.25:.0%}")  # 25%
```

- Older code formats the same way with `%` or `str.format`, recognise them :

```
print("Art: %5d, Price per Unit: %8.2f" % (art, price))
print("Art: {:5d}, Price per Unit: {:8.2f}".format(art, price))
```

demo13
formatspecs

SpiralTrain developer courses

Triple Quotes

- **Allow strings to span multiple lines :**
  - Including verbatim NEWLINEs, TABs, and any other special characters
- **Syntax consists of three consecutive single or double quotes :**

```
para_str = """this string is made up of several lines and non-printable
characters such as TAB ( \t ) and they will show up that way when displayed.
NEWLINEs within the string, whether explicitly given like this within the
brackets [ \n ], or just a NEWLINE within
the variable assignment will also show up."""
print(para_str)
```

- **Result :**

```
this string is made up of several lines and non-printable
characters such as TAB (         ) and they will show up that way when displayed.
NEWLINEs within the string, whether explicitly given like this within the
brackets [
 ], or just a NEWLINE within
the variable assignment will also show up.
```

demo14
triplequotes

SpiralTrain developer courses

Built-in String Methods

- **String methods are called with dot notation : `"albert".upper()`**
  - A string never changes, so each method returns a new string or a value

Method	Description
`strip()`	Removes whitespace from both ends, `lstrip()` and `rstrip()` from one
`lower()` / `upper()`	Converts the string to all lowercase or all uppercase
`split(sep)`	Splits the string into a list of words, on whitespace by default
`sep.join(words)`	Joins a list of strings into one, with `sep` between them
`count(sub)`	Returns how many times `sub` occurs in the string
`startswith(prefix)`	Returns `True` if the string starts with `prefix`, see also `endswith()`
`isalpha()`	Returns `True` if the string is not empty and holds only letters
`replace(old, new)`	Returns a copy with every `old` replaced by `new`
`find(sub)`	Returns the index of the first `sub`, or `-1` if it is not there

- **`len(s)` is a function rather than a method, and gives the number of characters**
- **All the others : `https://docs.python.org/3/library/stdtypes.html#string-methods`**

demo15
stringfunctions

SpiralTrain developer courses

Summary : Variables and Data Types

- **A variable is a name bound to an object :**
  - Created on first assignment with `=`, and can be annotated, as in `age: int = 25`
  - Every object has an identity, `id(x)`, a type, `type(x)`, and a value
- **Python is strongly typed :**
  - Converting between types is explicit, with `int()`, `float()`, `str()` and `bool()`
- **Numbers are `int`, `float` and `complex`; `bool` is a subtype of `int`**
- **Operators compute, compare and combine values :**
  - `==` compares values, `is` compares identities
- **Strings are immutable sequences of Unicode characters :**
  - Indexed and sliced with `[]`, and have methods such as `strip()` and `split()`
- **f-strings are the way to format text :**
  - `f"{price:8.2f}"` sets the width and the decimals of the value it shows

- Python Intro
- Variables and Data Types
- ▶ **Data Structures**
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

- Type Hinting
- Python Libraries

- Sequences
- Lists
- Accessing Lists
- Delete List Elements
- Multidimensional Lists
- List Operations
- List Functions and Methods
- Tuples
- Accessing Tuples
- Delete Tuple Elements
- Usage of Tuples
- Unpacking in for Loops
- Tuple Functions
- Sets
- Set Operations
- Dictionaries
- Accessing Values in Dictionary
- Properties of Dictionary Keys
- Annotating Collections
- Dictionary Functions and Methods

SpiralTrain developer courses

Sequences

- **Most basic data structure in Python is the sequence :**
  - Each element of a sequence is assigned a number, its position or index
  - First index is zero, the second index is one, and so forth
  - Python distinguishes sequences on the basis of mutability
- **Immutable sequences :**
  - Contain objects and content cannot change once created :
    - Objects may reference other objects, and those other objects may change
  - Examples are strings, tuples and bytes, which the IO module covers
- **Mutable sequences :**
  - Contain objects and content can be changed once created
  - The list is the example used everywhere
- **Operations that work on all sequences types :**
  - Indexing, slicing, multiplying, and checking for membership
- **Python has built-in functions for :**
  - Finding the length of a sequence and its largest and smallest elements

SpiralTrain developer courses

Lists

- **Lists are mutable sequences :**
  - List contains items separated by commas and enclosed within square brackets `[]`
  - The items of one list can be of different data types
- **Values are accessed using the index or slice operator `[ ]` and `[ : ]` :**
  - Indexes start at 0, then upwards and with end-1
  - Lists can be sliced, concatenated and so on
- **Lists respond to the `+` and * operators much like strings :**
  - Meaning is concatenation and repetition
  - Except that result is a new list, not a string
- **Lists respond to all sequence operations usable on strings :**

```
list1 = ["abcd", 786, 2.23, "john", True]
print(list1)  # Prints complete list
print(list1[0])  # Prints first element of the list
print(list1[1:3])  # Prints the 2nd and 3rd elements
print(list1 * 2)  # Prints list two times
print(list1[::-1])  # Prints complete list in reverse order
```

demo01
lists

SpiralTrain developer courses

Accessing Lists

- **Use index operator `[]` to access list at specific position :**
  - Counting runs from `0` at the front and from `-1` at the back
  - A slice runs up to but not including its second index

```
li = ["physics", "chemistry", "biology", "computer science"]
print("Value available at index 2 : ")
print(li[2])
li[2] = "mathematics"
print("New value available at index 2 : ")
print(li[2])
```

- **Elements can be added to a list with `append()` :**

```
list2 = [1, 2, 3]
list2.append(4)
list2.append(5)
print(list2)  # [1, 2, 3, 4, 5]
```

- **Elements can be added to a list at a position with `insert(index, item)` :**

```
list2.insert(0, 99)
print(list2)  # [99, 1, 2, 3, 4, 5]
```

SpiralTrain developer courses

Delete List Elements

- Remove a list element with `del` or `remove()`
- Use `del` statement if you know index of element to remove :

```
li = ["physics", "chemistry", "biology", "computer science"]
del li[2]
print("After deleting value at index 2 : ")
print(li)  # ['physics', 'chemistry', 'computer science']
```

- Use `remove()` if want to remove element by value :

```
li.remove("physics")
print("After deleting value 'physics' ")
print(li)  # ['chemistry', 'computer science']
```

- List itself can also be removed with `del` :

```
del list1
list1.append(100)  # gives NameError, name list1 not defined anymore
```

demo02
updatinglists

SpiralTrain developer courses

Multidimensional Lists

- A list can hold other lists, which gives a grid of rows and columns :

```
grid = [
    [1, 2, 3],
    [4, 5, 6],
]
print(grid[0])  # [1, 2, 3], the first row
print(grid[1][2])  # 6, row 1, column 2
print(len(grid), len(grid[0]))  # 2 3, rows and columns
```

- Each row is an ordinary list, so it can be changed in place or added to :

```
grid[0][0] = 10
print(grid)  # [[10, 2, 3], [4, 5, 6]]
grid.append([7, 8, 9])
print(grid)  # [[10, 2, 3], [4, 5, 6], [7, 8, 9]]
```

- Repeating a row with `*` repeats one list object, not a copy of it :

```
shared = [[0] * 3] * 2
shared[0][0] = 1
print(shared)  # [[1, 0, 0], [1, 0, 0]], both rows are one list
```

demo03
multidimensionallists

List Operations

Python Expression	Results	Description
`len([1,2,3])`	`3`	Length
`[1,2,3]+[4,5,6]`	`[1,2,3,4,5,6]`	Concatenation
`['go']*3`	`['go','go','go']`	Repetition
`3 in [1,2,3]`	`True`	Membership
`for x in [1,2,3]: print(x)`	`1 2 3`	Iteration

- A list also works as a stack : `append` puts an item on top, `pop` takes the top one off

```
stack = [3, 4, 5]
stack.append(6)
stack.append(7)
print(stack)  # [3, 4, 5, 6, 7]
print(stack.pop())  # 7
print(stack)  # [3, 4, 5, 6]
```

demo04
listsasstacks

SpiralTrain developer courses

List Functions and Methods

Function	Description
`len(list)`	Gives the total length of the list
`max(list)` / `min(list)`	Returns item from the list with max or min value
`list(seq)`	Converts a tuple into list
`list.append(obj)`	Appends object obj to list
`list.count(obj)`	Returns count of how many times obj occurs in list
`list.extend(seq)`	Appends the contents of seq to list
`list.index(obj)`	Returns the lowest index in list that obj appears
`list.insert(index, obj)`	Inserts object obj into list at offset index
`list.pop(index=-1)`	Removes and returns the item at an index; default is last
`list.remove(obj)`	Removes object obj from list
`list.reverse()`	Reverses objects of list in place
`list.sort(key=None, reverse=False)`	Sorts in place; options are keyword-only

demo05
listmethods

SpiralTrain developer courses

Tuples

- **Tuple is an immutable sequence of Python objects :**
  - Consist of a number of values separated by commas
  - Tuples are sequences, just like lists, but cannot be changed once created
- **Main differences between lists and tuples :**
  - Lists are enclosed in brackets `[ ]` and their contents and size can be changed
  - Tuples are enclosed in parentheses `( )` and cannot be updated
- **Create tuple by assigning comma-separated values to variable :**
  - Best practice is to put these comma-separated values between parentheses
  - An empty tuple is written as two parentheses containing nothing
  - For a tuple containing a single value include a comma, otherwise it is a single value

```
tup1 = ("physics", "chemistry", "biology", "computer science")
tup2 = (1, 2, 3, 4, 5)
tup3 = "a", "b", "c", "d"  # not recommended
tup4 = ()                  # empty tuple
tup5 = (50,)               # single element tuple
```

demo06
tuples

SpiralTrain developer courses

Accessing Tuples

- **All of the general sequence operations can be applied on tuples :**
  - Use square brackets for slicing along with index to obtain value, indices start at 0
  - Use the + and * operators for concatenation and repetition

```
tup = (1, 2, 3, 4, 5, 6, 7)
print(tup[0])  # 1
print(tup[1:5])  # (2, 3, 4, 5)
print(len(tup))  # 7
```

- **Tuples are immutable and can be thought of as read-only lists :**
  - Cannot update them or change values of tuple elements

```
tup[2] = 1000  # TypeError: 'tuple' object does not support item assignment
```

- **Portions of existing tuples can be used to create new tuples :**

```
tup1 = (12, 34.56)
tup2 = ("abc", "xyz")
tup3 = tup1 + tup2
print(tup3)  # (12, 34.56, 'abc', 'xyz')
```

- **Tuples can have other tuples or lists as elements**

demo07
tupleaccess

SpiralTrain developer courses

Delete Tuple Elements

- **Removing individual tuple elements is not possible :**
  - Another tuple with undesired elements discarded must be created
- **Entire tuple can be removed with `del` statement :**

```
tup = ("physics", "chemistry", "biology", "computer science")
print(tup)  # ('physics', 'chemistry', 'biology', 'computer science')
del tup
print("After deleting tup : ")
print(tup)
```

- **Result after deleting tup :**

```
Traceback (most recent call last):
  File "C:\course-work\tuples.py", line 5, in <module>
    print(tup)
          ^^^
NameError: name 'tup' is not defined
```

- **After `del tup`, the name is no longer bound; other references may keep the tuple alive**

SpiralTrain developer courses

Usage of Tuples

- **Tuples have many uses :**
  - Coordinate pairs, records from a database
- **Any set of comma-separated objects default to tuples :**
  - Even without parentheses around them
- **Tuples can be dictionary keys and set members if every item is hashable**
- **Tuples can make your code safer :**
  - Allows you to ”write-protect” data that does not need to be changed
- **A tuple can have lists as items :**
  - The items in the lists are mutable however
- **Tuple can be destructured and contents assigned to variables :**

```
t = ("Albert", "Einstein")
print(t)  # ('Albert', 'Einstein')
v, s = t
print(v)  # 'Albert'
print(s)  # 'Einstein'
```

demo08
destructurewithtuples

Unpacking in for Loops

- A `for` loop unpacks each item the same way, and the unpacking can nest :

```
rows = [(1,), (2,)]
for (number,) in rows:
    print(number)  # 1, then 2

records = [(1, ("Albert", "Einstein"))]
for number, (first, last) in records:
    print(number, first, last)  # 1 Albert Einstein
```

SpiralTrain developer courses

Tuple Functions

- Only functions and methods that do not modify tuple available

Function	Description
`len(tuple)`	Gives the total length of the tuple
`max(tuple)`	Returns item from the tuple with max value
`min(tuple)`	Returns item from the tuple with min value
`tuple(seq)`	Converts a list into tuple
`tup.index(i)`	Index of the first occurrence of i in tup
`tup.count(i)`	Total number of occurrences of i in tup

SpiralTrain developer courses

Sets

- **A set is an unordered collection with no duplicates :**
  - Created with curly braces `{ }` or with the `set()` constructor
  - `set()` takes one iterable, such as a list, and keeps each value once

```
s1 = {1, 1, 2, 2}
print(s1)  # {1, 2}
numbers = [3, 3, 4, 4]
s2 = set(numbers)
print(s2)  # {3, 4}
s2.add(5)
print(s2)  # {3, 4, 5}
print(4 in s2)  # True, a fast membership test
```

- **An empty set is written `set()`, because `{}` is an empty dictionary**
- **A `frozenset` is a set that cannot be changed after it is created :**

```
frozen = frozenset(numbers)
print(frozen)  # frozenset({3, 4})
```

demo09
setsandfrozensets

Set Operations

- Sets support the operations of set theory, each written as an operator :

```
physics = {"Einstein", "Bohr", "Curie", "Fermi"}
chemistry = {"Curie", "Rutherford", "Pauling"}
print("Curie" in chemistry)  # True
print(sorted(physics | chemistry))  # union, in either set
print(sorted(physics & chemistry))  # intersection, in both
print(sorted(physics - chemistry))  # difference, in physics only
print(sorted(physics ^ chemistry))  # in one of the two, not both
```

```
True
['Bohr', 'Curie', 'Einstein', 'Fermi', 'Pauling', 'Rutherford']
['Curie']
['Bohr', 'Einstein', 'Fermi']
['Bohr', 'Einstein', 'Fermi', 'Pauling', 'Rutherford']
```

- A set has no order, so `sorted()` turns it into a list to print it predictably
- Each operator has a method form that accepts any iterable :
  - `union()`, `intersection()`, `difference()` and `symmetric_difference()`

SpiralTrain developer courses

Dictionaries

- **Dictionaries consist of pairs of keys and values :**
  - Keys must be hashable; strings and numbers are common choices
  - Values can be any arbitrary Python object
- **Dictionaries are hash tables :**
  - Looking up a key is fast, however large the dictionary
  - Consist of key-value pairs separated by commas
- **Dictionaries are enclosed by curly braces `{ }` :**

```
dict1 = {"Albert": 2341, "Robert": 9102, "Enrico": 3258}
print(dict1)  # {'Albert': 2341, 'Robert': 9102, 'Enrico': 3258}
```

- **Values can be assigned and accessed using square brackets `[]` :**

```
dict1["Werner"] = 3490
dict1[2] = "Data"
print(dict1)  # {'Albert': 2341, 'Robert': 9102, 'Enrico': 3258, 'Werner': 3490, 2: 'Data'}
```

- **Dictionaries remember insertion order :**
  - Keys come out in the order in which they were first added

SpiralTrain developer courses

Accessing Values in Dictionary

- **Keys are unique within a dictionary while values may be the same :**
  - Values of a dictionary can be of any type
  - Keys must be hashable; a tuple qualifies only if all its items are hashable
- **Use square brackets along with the key to obtain its value :**

```
print(dict1["Robert"])  # 9102
print(dict1[2])  # Data
```

- **Dictionary can be updated by modifying an existing entry :**

```
dict1["Enrico"] = 9999
print(dict1["Enrico"])  # 9999
```

- **Dictionary entry can be deleted :**

```
del dict1["Albert"]
print(dict1["Albert"])  # KeyError
```

- **Entire contents of dictionary is cleared with `clear()` :**
  - Entire dictionary can be deleted in single operation with del

demo10
dictionaries

SpiralTrain developer courses

Properties of Dictionary Keys

- **Keys must be hashable :**
  - Their hash stays stable, and equal keys must have equal hashes
  - Strings and numbers qualify; lists and dictionaries do not
  - A tuple is hashable only when every item is hashable
- **Values can be any object; assigning an equal key replaces its value**

```
extensions = {("Albert", "Einstein"): 2341}
print(extensions[("Albert", "Einstein")])  # 2341
```

- **An immutable tuple can contain an unhashable list :**

```
hash(("Albert", []))  # TypeError: unhashable type: 'list'
```

SpiralTrain developer courses

Annotating Collections

- Each built-in collection is annotated with the type of what it holds :

```
scores: list[int] = [10, 20, 30]
names: set[str] = {"Alice", "Bob"}
extensions: dict[str, int] = {"Albert": 2341, "Robert": 9102}
```

- A dictionary takes two types, the key first and then the value
- A tuple is annotated position by position, because its length is fixed :

```
point: tuple[int, int] = (3, 4)
row: tuple[str, int, float] = ("Audi", 2019, 52643.0)
```

- An ellipsis means any number of one type :

```
readings: tuple[float, ...] = (1.5, 2.5, 3.5)
```

- Annotate where the collection is homogeneous, which most are :
  - A deliberately mixed list such as `['abcd', 786, 2.23]` has no single element type, and is better left bare than annotated `list[object]`

SpiralTrain developer courses

Dictionary Functions and Methods

Function	Description
`len(dict)`	Gives the total length, or nr. of items, in the dictionary
`str(dict)`	Produces a printable string representation of a dictionary
`fromkeys()`	Create new dictionary with keys from seq, values set to value
`dict.clear()`	Removes all elements of dictionary dict
`dict.copy()`	Returns a shallow copy of dictionary dict
`dict.get(key)`	For key key, returns value or default if key not in dictionary
`dict.popitem()`	Removes and returns the (key, value) pair added last
`dict.items()`	Returns a view of dict's (key, value) tuple pairs
`dict.keys()`	Returns a view of dictionary dict's keys
`dict.update(dict2)`	Adds dictionary dict2's key-values pairs to dict
`dict.values()`	Returns a view of dictionary dict's values

demo11
dictionarymethods

SpiralTrain developer courses

Summary : Data Structures

- **Sequences are most basic data structure :**
  - Each element of a sequence is assigned a number, its position or index
  - First index is zero, the second index is one, and so forth
- **Lists are mutable sequences :**
  - Contain items separated by commas and enclosed within square brackets `[]`
  - Items can be of different data types, and lists can hold other lists
- **Tuples are immutable sequences :**
  - Enclosed in parenthesis ( ) instead of brackets
  - Consist of a number of values separated by commas
- **Sets are unordered collections with no duplicates :**
  - Fast membership tests, and union, intersection and difference
  - Set is mutable and `frozenset` is immutable
- **Dictionaries map keys to values, in insertion order :**
  - Pairs of keys and values, separated by a comma and a colon
  - Enclosed by curly braces `{ }`, values accessed with square brackets `[]`

- Python Intro
- Variables and Data Types
- Data Structures
- ▶ **Control Flow**
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

- Type Hinting
- Python Libraries

- Control Flow Constructs
- if Statement
- if Example
- else Statement
- elif Statement
- Nested if...elif...else
- while Loop
- Infinite while Loop
- for Loop
- Iterating by Sequence Index
- Iterating with enumerate
- break and continue Statements
- Loop with else Combination
- pass Statement
- match Statement

Control Flow Constructs

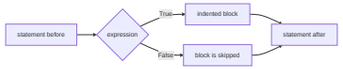
- **Statements run one after the other unless a control flow construct intervenes :**
  - Branching constructs choose which block runs
  - Iteration constructs repeat a block
- **Branching statements :**
  - `if`, `if...else` and `elif`
  - `match`, which compares one value against a series of patterns
- **Iteration statements :**
  - `while`, which repeats as long as a condition holds
  - `for`, which takes each item of a sequence in turn
- **Statements that steer a loop or fill a gap :**
  - `break`, `continue`, a loop's `else`, and `pass`

if Statement

- `if` tests an expression and runs a block only when it is true :
  - The expression is often a comparison, such as `count > 0`
  - The block is skipped when the expression is false
- Syntax `if` statement :

```
if expression:
    statement(s)
```

- Expression is evaluated first :
  - If it is true, the indented block runs
  - Otherwise execution continues with the statement after the block



- Values other than `True` and `False` can also be tested :
  - `0`, `""`, empty collections and `None` count as false, everything else as true
- Grouping statements :
  - Statements indented by the same amount form one block
  - Python uses indentation, four spaces by convention, as its way of grouping

SpiralTrain developer courses

if Example

```
var1 = 100
if var1:
    print("1 - Got a true expression value")
    print(var1)

var2 = 0
if var2:
    print("2 - Got a true expression value")
    print(var2)
print("Good bye!")
```

- **Result :**

```
1 - Got a true expression value
100
Good bye!
```

- **In case `if` has only one line it may go on same line :**

```
if expression == 1: print("Value of expression is 1")
```

SpiralTrain developer courses

else Statement

- **Is combined with an `if` statement :**
  - Contains the block that runs when the `if` expression is false
- **`else` is optional :**
  - At most one `else` can follow an `if`, and it comes last
- **Syntax `if...else` statement :**

```
if expression:
    statement(s)
else:
    statement(s)
```

- **Example reading input with `input` and testing it with `endswith` :**

```
name = input("What is your name? ")
if name.endswith("Gumby"):
    print("Hello, Mr. Gumby")
else:
    print("Hello, stranger")
```

demo02
else

SpiralTrain developer courses

elif Statement

- **Tests several expressions in turn :**
  - The block of the first expression that is true runs, and the rest are skipped
- **`elif` is optional :**
  - Any number of `elif` branches may follow an `if`
- **Syntax of the `if...elif` statement :**

```
if expression1:
    statement(s)
elif expression2:
    statement(s)
elif expression3:
    statement(s)
else:
    statement(s)
```

- **Where a chain of `elif` tests one value, `match` often reads better :**
  - Covered at the end of this module

demo03
elif

SpiralTrain developer courses

Nested if...elif...else

- An `if` statement can sit inside the block of another :
  - The inner condition is only tested once the outer one is true
- Syntax nested `if...elif...else` :

```
if expression1:
    statement(s)
    if expression2:
        statement(s)
    elif expression3:
        statement(s)
    else:
        statement(s)
elif expression4:
    statement(s)
else:
    statement(s)
```

- Each level of nesting adds four spaces :
  - More than two levels usually reads better as `and`, or as a function

demo04
nested_if

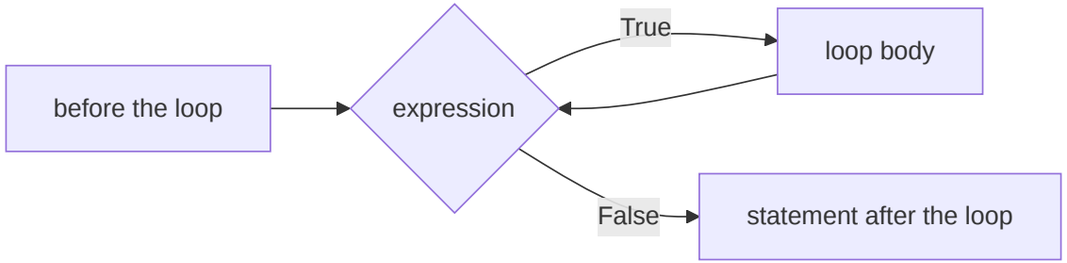
SpiralTrain developer courses

while Loop

- **Repeats a block for as long as an expression is true :**
  - The expression is tested before every pass, including the first

```
while expression:
    statement(s)
```

- **The loop ends when the expression is false :**
  - Execution continues with the statement after the loop



- **If the expression never becomes false, the loop never ends :**
  - Sometimes that is the intent, as in a server or a game that waits for input
  - Such a loop is written `while True:` and left with `break`, covered below
- **A body that must run at least once is written the same way :**
  - `while True:`, then the body, then `if done:` with `break` under it

demo05 while

SpiralTrain developer courses

Infinite while Loop

```
while True:  # the condition never becomes false
    num = input("Enter a number: ")
    print("You entered:", num)
print("Good bye!")  # never reached
```

- **Result :**

```
Enter a number: 20
You entered: 20
Enter a number: 3
You entered: 3
Enter a number: Traceback (most recent call last):
  File "06_infinitewhile.py", line 2, in <module>
    num = input("Enter a number: ")
KeyboardInterrupt
```

- **A loop written this way continues until interrupted :**
  - Press Ctrl+C to stop the program

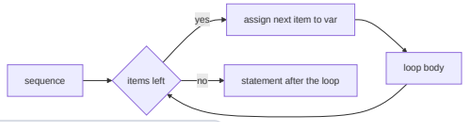
demo06
infinitewhile

for Loop

- **Repeats a block for each item of a sequence :**
  - Iterates over any sequence or iterable, such as a list, a string or a range
- **Syntax :**

```
for var in sequence:
    statement(s)
```

- **How a `for` loop works :**
  - Each item is assigned to variable `var` in turn
  - The block runs once per item until the sequence is exhausted




```
words = ["one", "way", "ticket", "to", "mars"]
for word in words:
    print(word, end=" ")
print()
list1 = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
for number in list1:
    print(number)
```

demo07
for

SpiralTrain developer courses

Iterating by Sequence Index

- **A sequence can be walked by index, using `len()` and `range()` :**

```
planets = ["venus", "mars", "jupiter"]
for index in range(len(planets)):
    print("Current planet :", planets[index])
```

- **This works, and is what a loop over positions looks like in many languages :**
  - `len()` gives the number of elements, `range()` the sequence of positions
  - It is worth recognising, because you will meet it in other people's code
- **In Python it is rarely what you want :**
  - The index is only used to look the element back up again
  - A `range` that drifts out of step with the sequence is a common bug
- **When the element is all you need, iterate the sequence itself :**

```
for planet in planets:
    print("Current planet :", planet)
```

SpiralTrain developer courses

Iterating with enumerate

- When the index is genuinely needed, `enumerate` yields both :

```
for index, planet in enumerate(planets):
    print(index, planet)
```

```
0 venus
1 mars
2 jupiter
```

- The two can never fall out of step, because they come from one call
- Counting starts at zero unless a `start` is given :

```
for number, planet in enumerate(planets, start=1):
    print(number, planet)
```

- The pattern generalises :
  - `enumerate(seq)` for position and element
  - `zip(a, b)` for two sequences walked in step, covered in Comprehensions
  - `dict.items()` for a dictionary's keys and values together

demo08
loopbyindex

SpiralTrain developer courses

break and continue Statements

- `break` leaves the loop immediately :
  - Execution continues with the statement after the loop
- `continue` skips the rest of the current iteration :
  - Control moves directly to the next pass of the loop
- Both statements work inside `while` and `for` loops

```
for letter in "Python":
    if letter == "h":
        continue
    print("Current Letter :", letter)  # letter 'h' is not printed
print("After first loop!")
for letter in "Programming":
    if letter == "i":
        break
    print("Current Letter :", letter)  # loop stops on letter 'i'
print("After second loop!")
```

demo09
breakandcontinue

SpiralTrain developer courses

Loop with else Combination

- **An `else` clause can be attached to a loop :**
  - In a `for` loop, the `else` block runs when the sequence is exhausted
  - In a `while` loop, the `else` block runs when the condition becomes false
  - In both cases, the `else` block is skipped if the loop was left via `break`
- **In the following example :**
  - The loop searches for a perfect square downwards from 99
  - When 81 is found, `break` leaves the loop and skips the `else` block

```
from math import sqrt
for n in range(99, 80, -1):
    root = sqrt(n)
    print(root)
    print(int(root))
    if root == int(root):
        print(f"Perfect square is {n}")
        break
else:
    print("Didn't find it!")
```

demo10
for_else

SpiralTrain developer courses

pass Statement

- **Used when Python's syntax requires a statement but no action is needed :**
  - A null operation; nothing happens when it executes
  - Commonly used as a placeholder in stubs or empty branches
- **When a code block is under construction :**
  - `pass` keeps the file syntactically valid until code is written

```
for letter in "Python":
    if letter == "h":
        pass
        print("This is pass block")
    print("Current Letter :", letter)
print("Good bye!")
```

demo11
passstatement

SpiralTrain developer courses

match Statement

- **Python 3.10 added structural pattern matching, `match` and `case` :**
  - It compares a value against a series of patterns and runs the first that fits
  - Older Python has no such statement, which is why chains of `elif` are common

```
match command.split():
    case ["quit"]:  # a literal pattern
        return "leaving"
    case ["go", direction]:  # captures the second word
        return f"walking {direction}"
    case ["take" | "get", item] if item != "moon":  # or-pattern with a guard
        return f"picking up the {item}"
    case _:  # the wildcard, matched last
        return "I do not understand"
```

- **It matches the shape of the data as well as its value :**
  - A pattern can bind names out of a sequence, as `direction` and `item` do
  - `case _` is the catch-all, and belongs last
- **There is no fall through between cases, so a case needs no `break` to end it; a `break` inside a case still leaves the loop around the `match`**

demo12
matchstatement

SpiralTrain developer courses

Summary : Control Flow

- **Branching statements choose which code runs :**
  - `if`, `if...else` and `elif` test conditions in turn
  - `match` compares data against a series of patterns
- **Iteration statements repeat a block of code :**
  - `while` repeats as long as a condition holds
  - `for` iterates over each item of a sequence
- **Statements that steer a loop :**
  - `break` exits the loop early
  - `continue` skips to the next iteration
  - An `else` clause runs when the loop finishes without a `break`
- **The `pass` statement is a no-op placeholder where a statement is required**

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- ▶ **Functions**
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

- Type Hinting
- Python Libraries

- Functions
- Function Syntax
- Annotating a Function
- Calling Functions
- Passing Object References
- Overwriting References
- Mutating Versus Rebinding
- Function Arguments
- Keyword Arguments
- Default Arguments
- Variable Length Arguments
- Lambda Functions
- Functions in Dictionaries
- return Statement
- Scope of Variables
- Example Variable Scope

Functions

- **Named blocks of reusable code :**
  - Python includes many built-in functions such as `print()`, `input()`, `len()`
  - Programs define their own functions with `def`
- **Functions provide modularity and code reuse :**
  - A task is written and tested once, then called from multiple places
- **Defining a function :**
  - Starts with `def`, the function name, and parentheses for parameters
  - The header line ends with a colon `:`, and the body is indented
- **Exiting a function :**
  - The `return` statement leaves the function and passes a value back
  - A function without a `return`, or a bare `return`, returns `None`

Function Syntax

- **The first statement can be a documentation string (docstring) :**
  - Explains what the function does and what parameters it expects
  - Documenting tools such as `pdoc` extract docstrings into HTML

```
def function_name(parameter: type) -> return_type:
    """function docstring"""
    statement(s)
    return expression
```

- **Positional parameters :**
  - By default, arguments are passed in the order they are defined
- **Example of a simple function :**
  - Accepts a string and prints it to standard output

```
def show_message(message: str) -> None:
    """Print the passed string."""
    print(message)
```

SpiralTrain developer courses

Annotating a Function

- **Each parameter takes a colon and the type it expects :**
  - The result takes an arrow and the type it returns
  - A function that returns nothing is annotated `-> None`

```
def area(width: float, height: float) -> float:
    return width * height


def show_message(message: str) -> None:
    print(message)
```

- **The annotation is documentation the interpreter carries with the function :**
  - Nothing is converted and nothing is checked while the program runs
  - Passing the wrong type still works, right up until an operation fails
- **What it buys is everything that reads the signature :**
  - The editor, a checker, and the next person to read the code
- **A collection parameter is annotated like any other collection :**

```
def average(values: list[float]) -> float:
    return sum(values) / len(values)
```

SpiralTrain developer courses

Calling Functions

- **A function is invoked by its name followed by parentheses :**
  - Any required arguments are placed inside the parentheses
- **Functions must be defined before they are called :**
  - Python executes line by line, so definitions must come before use
- **Calling `show_message()` :**

```
show_message("First call to user defined function!")
show_message("Again second call to the same function")
show_message()  # results in error
```

- **Output :**

```
First call to user defined function!
Again second call to the same function
Traceback (most recent call last):
  File "01_showmessage.py", line 10, in <module>
    show_message()
TypeError: show_message() missing 1 required positional argument: 'message'
```

demo01
showmessage

SpiralTrain developer courses

Passing Object References

- **Python passes arguments by object reference :**
  - The parameter inside the function is bound to the same object as the caller's argument
  - Both names refer to the identical object in memory
- **Mutating an object affects the caller :**
  - Modifying a mutable object (such as a list) in place is visible outside
  - Immutable objects (such as integers or strings) cannot be modified in place

```
def change_me1(mylist: list[int]) -> None:
    mylist.append(40)
    print("List inside the function change_me1:", mylist)
```

- **Calling `change_me1` :**

```
my_list_outer = [10, 20, 30]
change_me1(my_list_outer)
print("List outside the function:", my_list_outer)
```

- **The caller sees the item added to the list :**

```
List inside the function change_me1: [10, 20, 30, 40]
List outside the function: [10, 20, 30, 40]
```

demo02
objectreferences

SpiralTrain developer courses

Overwriting References

- **Rebinding a parameter does not affect the caller :**
  - Assigning to the parameter name creates a new local binding
  - The caller's variable continues to refer to the original object

```
def change_me2(mylist: list[int]) -> None:
    mylist = [1, 2, 3, 4]  # rebinds the local name
    print("Values inside the function change_me2:", mylist)
```

- **Calling `change_me2` :**

```
my_list_outer = [10, 20, 30]
change_me2(my_list_outer)
print("Values outside the function:", my_list_outer)
```

- **The caller's list is unchanged :**

```
Values inside the function change_me2: [1, 2, 3, 4]
Values outside the function: [10, 20, 30]
```

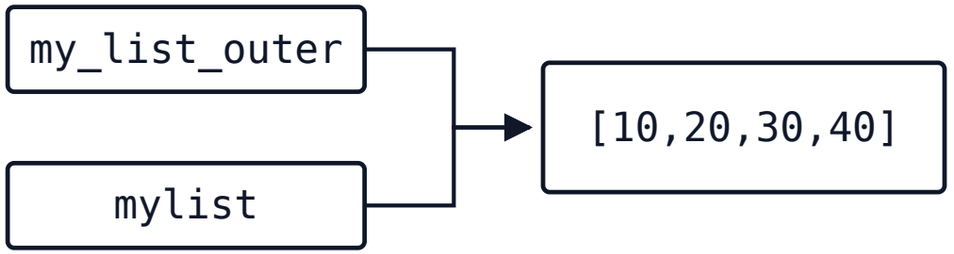
demo03 overwritingreferences

demo04 swap

Mutating Versus Rebinding

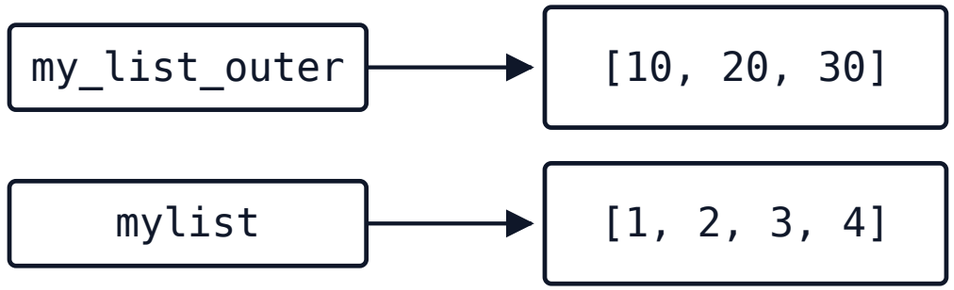
- **The two previous slides differ in one character, and that changes everything :**

```
mylist.append(40)
```



one object, two names, the caller sees the change

```
mylist = [1, 2, 3, 4]
```



the name is rebound, the caller sees nothing

SpiralTrain developer courses

Function Arguments

- **Python supports several styles of function arguments :**
- **Positional arguments :**
  - Passed to a function in order matching the parameter list
- **Keyword arguments :**
  - Passed by name (`name=value`), allowing arguments to be given in any order
- **Default arguments :**
  - Parameters that take a default value when omitted by the caller
- **Variable-length arguments :**
  - Accept an arbitrary number of positional or keyword arguments
- **Python does not support function overloading :**
  - Defining the same function name multiple times with different signatures replaces the earlier definition

demo05
power1

SpiralTrain developer courses

Keyword Arguments

- **Function definition :**

```
def print_info(name: str, age: int) -> None:
    """Print the name and age passed to this function."""
    print("Name:", name)
    print("Age:", age)
```

- **Calling with keyword arguments :**

```
print_info(age=55, name="Albert")
```

- **Output :**

```
Name: Albert
Age: 55
```

- **Calling code specifies arguments by name :**
  - Positional order does not need to be followed
  - Positional arguments must appear before keyword arguments

demo06
keywordarguments

SpiralTrain developer courses

Default Arguments

- Default values provide a fallback when an argument is omitted :

```
def print_info(name: str, age: int = 35) -> None:
    """Print name and age, defaulting age to 35."""
    print("Name:", name)
    print("Age:", age)
```

- Calling with and without the default argument :

```
print_info(age=55, name="Albert")
print_info(name="Werner")
```

- Output :

```
Name: Albert
Age: 55
Name: Werner
Age: 35
```

- Parameters without defaults cannot follow parameters with defaults

demo07
defaultarguments

Variable Length Arguments

- `*args` gathers extra positional arguments into a tuple :
  - Commonly used when a function accepts any number of items
- `**kwargs` gathers extra keyword arguments into a dictionary :
  - Commonly used for configuration or passing options through

```
def show_arg_lists(arg: int, *args: int, **kwargs: int) -> None:
    """Demonstrate positional and keyword variable-length arguments."""
    print("arg   :", arg)
    print("args  :", args)
    print("kwargs:", kwargs)


show_arg_lists(1, 2, 3, 4, 5, x=1, y=2)
show_arg_lists(33, x=8, y=9)
show_arg_lists(44)
```

demo08
variableargumentlist

SpiralTrain developer courses

Lambda Functions

- **The `lambda` keyword creates a small anonymous function inline :**
  - Has no `def` statement and no name of its own
  - The body is a single expression evaluated and returned automatically
  - Cannot contain statements, loops or assignments
- **Best framed as a small function passed to another function :**
  - `sorted(key=...)` is the classic use case

```
words = ["banana", "pie", "Washington", "book"]

# Sort by last letter
by_last = sorted(words, key=lambda w: w[-1])
print("By last letter  :", by_last)

# Sort case-insensitively
by_case = sorted(words, key=lambda w: w.lower())
print("Case-insensitive:", by_case)
```

- **Output :**

```
By last letter  : ['banana', 'pie', 'book', 'Washington']
Case-insensitive: ['banana', 'book', 'pie', 'Washington']
```

demo09
lambdafunctions

Functions in Dictionaries

- **A function is a value : it can be stored in a dictionary and called from there**
- **A dictionary from keys to functions is a dispatch table :**
  - Look the function up by its key, then call it : `converters[unit](value)`
  - Replaces a long `if` / `elif` chain; the values can be named functions or lambdas

```
converters = {
    "C": to_celsius,
    "F": to_fahrenheit,
    "K": lambda value: value + 273.15,
}

unit = "C"
print(converters[unit](212))  # look up the function, then call it

for unit, convert in converters.items():
    print(unit, convert(100))
```

```
100.0
C 37.77777777777778
F 212.0
K 373.15
```

demo10
functionsindictionaries

SpiralTrain developer courses

return Statement

- **Exits a function and optionally passes a value back to the caller :**
  - A function without a `return`, or a bare `return`, returns `None`
- **Function definition :**

```
def summation(arg1: int, arg2: int) -> int:
    """Add both parameters and return the sum."""
    tot = arg1 + arg2
    print("Inside the function :", tot)
    return tot
```

- **Calling `summation` :**

```
total = summation(10, 20)
print("Outside the function :", total)
```

- **Output :**

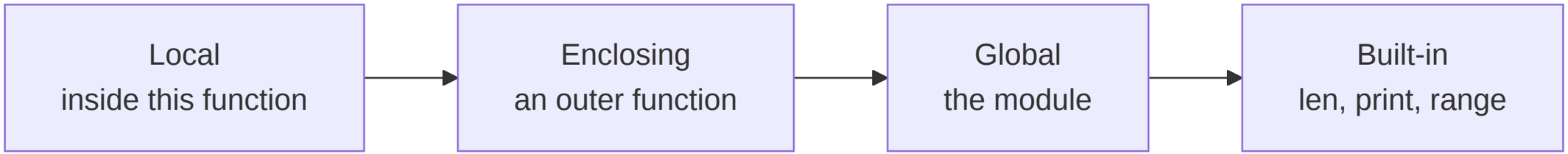
```
Inside the function : 30
Outside the function : 30
```

demo11
returnstatement

SpiralTrain developer courses

Scope of Variables

- **Scope determines where an identifier can be accessed :**
  - Variables created inside a function are local to that function
  - Variables created outside any function belong to the module's global scope
- **Python looks up names in four scopes, in LEGB order :**



- **The first scope that contains the name wins; the rest are not searched**
- **Local variables :**
  - Exist only while the function is executing
  - Cannot be read or modified from outside the function
- **Global variables :**
  - Can be read from anywhere within the module
  - Rebinding a global name inside a function requires `global`
  - Mutating the object it names, such as calling `items.append(x)`, does not

SpiralTrain developer courses

Example Variable Scope

- Global variables are shadowed by local bindings unless declared `global` :

```
x = 50
y = 50

def func1() -> None:
    x = 2
    print("Local x   :", x)

def func2() -> None:
    global y
    y = 2

func1()
print("x is still:", x)  # 50: global x unchanged
func2()
print("y is now  :", y)  # 2: global y modified
```

demo12
variablescope

SpiralTrain developer courses

Summary : Functions

- **Functions are reusable blocks of code defined with `def` :**
  - Parameters and return types are documented with type annotations
- **Python passes arguments by object reference :**
  - Mutating an argument affects the caller; rebinding a parameter name does not
- **Flexible argument handling :**
  - Positional, keyword, default values, `*args` and `**kwargs`
- **Small anonymous functions use `lambda` :**
  - Ideal as short key functions passed to `sorted()`
- **Functions are values :**
  - A dictionary of functions is a dispatch table, `table[key](argument)`
- **Variables follow LEGB scope lookup :**
  - Local, Enclosing, Global and Built-in scopes

exercise05
Functions

SpiralTrain developer courses

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- ▶ **Modules**
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

- Type Hinting
- Python Libraries

- Modules
- import Statement
- from...import Statement
- Locating Modules
- Using Modules
- dir Function
- Packages in Python
- Explicit Import of Package Modules
- Implicit Import of Package Modules
- Test Harness
- Running Packages with python -m
- Installing Packages
- Virtual Environments

Modules

- **Modules organize Python definitions across separate files :**
  - Grouping related functions and classes makes code easier to read and maintain
  - Code defined in a module can be imported and reused across multiple programs
- **A module is simply a file containing Python code :**
  - Definitions include functions, classes, variables and runnable statements
  - The module name is the file name without the `.py` extension
- **For example, `test.py` defines `print_func` :**

```
def print_func(par: str) -> None:
    print("Hello :", par)
```

- **Any Python file can import `test.py` and call its functions**

SpiralTrain developer courses

import Statement

- **The `import` statement makes a module available to the calling file :**
  - Python searches for the module in the directories listed on `sys.path`
  - The module's top-level code runs once when imported for the first time
- **Syntax of the `import` statement :**

```
import module1[, module2[, ... moduleN]]
```

- **Accessing definitions through the module prefix :**
  - Functions, classes and variables are prefixed with the module name

```
import test  # Now you can call defined function

test.print_func("pipo")  # Hello : pipo
```

- **Modules can be renamed upon import using `as` :**

```
import test as t

t.print_func("pipo")  # Hello : pipo
```

demo01
importmodule

from...import Statement

- `from...import` makes specific names accessible directly without prefix :

```
from modname import name1[, name2[, ... nameN]]
```

- For example, importing `sqrt` from module `math` :

```
from math import sqrt
```

- The statement imports only the named items into the current namespace :
  - The module object itself is not bound, only `sqrt`
  - The function is called directly without a module prefix :

```
print(sqrt(9))  # 3.0
```

- Importing all names with `*` is also possible :

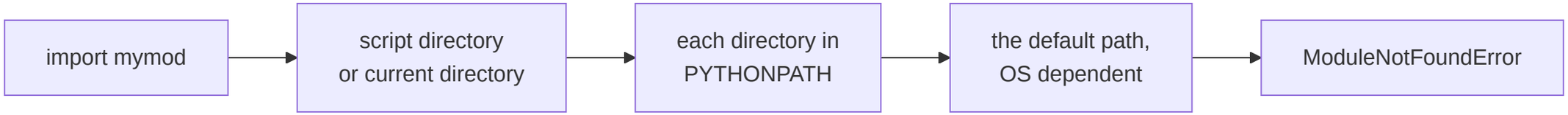
```
from math import *
```

- Wildcard imports should be used sparingly :
  - They clutter the namespace and risk name collisions

demo02
usingfromimport

Locating Modules

- For modules found through `sys.path`, the normal search order is :



- The first directory depends on how Python starts :
  - Running a script uses its directory; the REPL and `python -m` use the current directory
- Inspect the search path with `sys.path` after `import sys`
- `PYTHONPATH` environment variable :
  - Extra directories inserted after that first directory on startup
  - Follows standard operating system path list conventions
- Setting `PYTHONPATH` in Windows Command Prompt (CMD) :

```
set PYTHONPATH=C:\course\modules
```

- Setting `PYTHONPATH` on UNIX :

```
export PYTHONPATH=/usr/local/lib/python
```

demo03
pythonpath

Using Modules

- Any Python file can be imported by other files on `sys.path` :
  - `import mod` imports the module; `from mod import ...` imports specific names

```
# Filename: testmodules.py
def sayhi() -> None:
    print("Hi, this is testmodules speaking.")


version = "0.1"
```

- Importing `testmodules` and calling its functions :

```
import testmodules

testmodules.sayhi()
print(testmodules.version)

from testmodules import sayhi, version

sayhi()
```

demo04
usingmodules

dir Function

- **Returns a sorted list of attribute names accessible on an object or module :**
  - Lists module variables, functions, classes and imported names
  - Useful in the interactive REPL to explore an unfamiliar library
- **Inspecting a module :**

```
import math

print(dir(math)[:6])
```

- **Result :**

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos']
```

- **Special attributes start and end with double underscores, such as `__name__` and `__doc__`**

demo05
usingdir

Packages in Python

- **A package organizes multiple modules together under a shared directory :**
  - Provides a hierarchical namespace using dotted module names
  - Prevents naming collisions between modules in different packages
- **A directory is recognised as a package by having an `__init__.py` file :**

```
cameras/
├── __init__.py     marks the directory as a package
├── canon.py        defines canon_info()
├── leika.py        defines leika_info()
└── nikon.py        defines nikon_info()
```

- **For example, `canon.py` defines a function :**

```python
def canon_info() -> None:
    print("Canon camera")
```

- **`__init__.py` runs when the package is imported :**
  - It can be empty or used to expose package-level exports

SpiralTrain developer courses

Explicit Import of Package Modules

- When `__init__.py` is empty, modules inside the package are imported explicitly :

```
import cameras.canon
import cameras.leika
import cameras.nikon
```

- Functions inside the modules are called using the full dotted path :

```
cameras.nikon.nikon_info()
cameras.canon.canon_info()
cameras.leika.leika_info()
```

- Dotted names follow the file structure :
  - `package.module.function()`
  - Explicit imports keep it clear which module each function originates from

demo06
explicitimport

SpiralTrain developer courses

Implicit Import of Package Modules

- `__init__.py` can import names from its internal modules to re-export them :

```
from cameras.canon import canon_info
from cameras.leika import leika_info
from cameras.nikon import nikon_info
```

- Callers can then import the package directly without naming submodules :

```
import cameras

cameras.nikon_info()
cameras.canon_info()
cameras.leika_info()
```

- Callers can also import the re-exported functions directly :

```
from cameras import canon_info, leika_info, nikon_info

nikon_info()
canon_info()
leika_info()
```

demo07
implicitimport

Test Harness

- **Python sets `__name__` to `"__main__"` when run directly, or the module name when imported :**
  - The `if __name__ == "__main__":` guard separates library code from execution
  - Functions and classes can be imported without unwanted side effects
  - Exercise 5 uses this guard so `wordle.py` can be imported in the interpreter without starting a game

```
def sayhi() -> None:
    print("Hi, this is testmodules speaking.")


def main() -> None:
    sayhi()


if __name__ == "__main__":
    main()
```

- **Direct execution runs `main()`; importing `testharness` defines `sayhi()` without running `main()`**

demo08
testharness

SpiralTrain developer courses

Running Packages with python -m

- **Packages can be executed as a script using `python -m package_name` :**
  - Python executes `__main__.py` inside the package directory
  - Serves as the package entry point, like `if __name__ == "__main__":` in a single file
- **Package layout with `__main__.py` :**
  - `cameras3/` contains `__init__.py`, `__main__.py`, `canon.py`, `nikon.py`

```python
# cameras3/__main__.py
from cameras3 import canon_info, nikon_info


def main() -> None:
    print("Cameras package entry point:")
    canon_info()
    nikon_info()


if __name__ == "__main__":
    main()
```

- **Run from the terminal with `uv run python -m cameras3`**

demo09
package_main

SpiralTrain developer courses

Installing Packages

- **Third-party packages are hosted on PyPI and installed with `uv add` :**
  - `uv add package` installs a package and records it in `pyproject.toml`
  - `uv add --dev package` adds a development tool, such as `pytest`
  - `uv remove package` uninstalls a dependency
- **Two files track dependencies and belong in version control :**
  - `pyproject.toml` declares the project's direct requirements
  - `uv.lock` records the exact resolved versions for reproducibility
- **Anyone with the project can reproduce the exact environment :**

```
uv add requests
uv add "requests>=2.28,<3"
uv add --dev pytest
uv sync          # install exactly what uv.lock says
uv remove requests
```

SpiralTrain developer courses

Virtual Environments

- **Virtual environments isolate dependencies per project :**
  - Each project has its own libraries and Python executable
  - Different projects can require different versions without conflict
- **`uv` manages virtual environments automatically :**
  - `.venv/` is created on the first `uv run` or `uv add`
  - Packages are installed into `.venv/` without touching system Python
- **There is no environment to activate manually :**
  - `uv run` puts the project's `.venv` in front for the command

```
myproject/
├── pyproject.toml   dependencies declared for this project
├── uv.lock          pinned versions of all installed packages
└── .venv/           created on the first uv run or uv add
```

- **Run code through `uv run`, which uses the project's interpreter :**

```
uv run python app.py
```

- **Without `uv`, environments are made by `python -m venv .venv` and activated by hand**

SpiralTrain developer courses

Summary : Modules

- **Modules organize code into reusable files :**
  - A module is a `.py` file defining functions, classes and variables
  - `import` and `from ... import` bring names into the calling file
- **Python locates modules using `sys.path` :**
  - Script directory (current directory for REPL or `-m`), then `PYTHONPATH` and defaults
  - `dir()` lists the names defined or imported in a module
- **Packages group modules into directories with `__init__.py` :**
  - Modules inside packages use dotted import paths
  - `python -m package` runs a package via its `__main__.py`
- **Executing versus importing :**
  - `if __name__ == "__main__":` guards code that only runs when executed directly
- **Dependency and environment management :**
  - `uv add` installs packages into an automatic `.venv`
  - `uv run` executes commands with the project's environment active

exercise06
Modules

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- ▶ **Comprehensions**
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

- Type Hinting
- Python Libraries

- List Comprehensions
- Syntax List Comprehensions
- Filtered List Comprehension
- Nested List Comprehensions
- Comprehension Features
- Zip Function
- Dictionary Construction with zip
- Dictionary Comprehensions
- Dictionary from Keys
- Set Comprehensions
- Generator Expressions
- Built-ins that Take a Function
- Recognising map and filter
- Comprehensions and map
- Recognising reduce

SpiralTrain developer courses

List Comprehensions

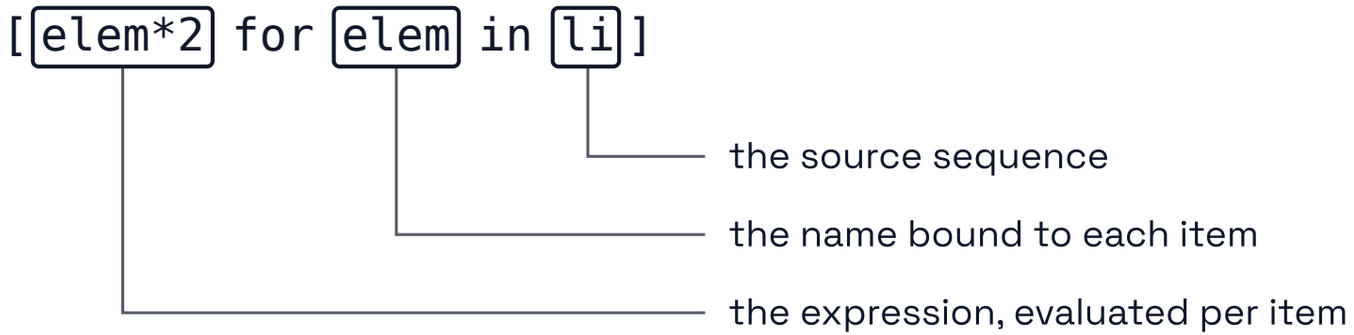
- A comprehension builds a new list by evaluating an expression for each item.
- Read the `for` part first, then the expression on its left.

```
numbers: list[int] = [1, 2, 3, 4, 5]
doubled = [number * 2 for number in numbers]
print(doubled)  # [2, 4, 6, 8, 10]
labels = [str(number) for number in numbers]
print(labels)  # ['1', '2', '3', '4', '5']
```

demo01
list_comprehensions

Syntax List Comprehensions

- **Syntax of a list comprehension may involve :**
  - `for` loop, an `in` operation and an `if` statement
- **Python's notation is `[ expression for name in list ]` :**



- **Reads as the loop that would otherwise be written out in full :**

```
li = [3, 6, 2, 7]
li_new = [elem * 2 for elem in li]  # [6, 12, 4, 14]
```

- **`if` statement acts as a filter and is optional :**

```
grades = [12, 0, 18]
print([x - 10 for x in grades if x > 0])  # [2, 8]
```

SpiralTrain developer courses

Filtered List Comprehension

- An optional `if` keeps only the items that satisfy its condition.
- These two forms produce the same list :

```
numbers = [1, 2, 3, 4, 5, 6]
labels = []
for number in numbers:
    if number % 2 == 0:
        labels.append(str(number))
print(labels)  # ['2', '4', '6']

labels = [str(number) for number in numbers if number % 2 == 0]
print(labels)  # ['2', '4', '6']
```

demo02
comprehension_with_predicate

Nested List Comprehensions

- The inner comprehension creates one row; the outer one creates the rows.
- Keep nesting short enough to read at a glance.

```
list_a = ["A", "B"]
list_b = ["C", "D"]
rows = [[x + y for x in list_a] for y in list_b]
print(rows)  # [['AC', 'BC'], ['AD', 'BD']]
```

- A comprehension can also have two `for` clauses, and the second runs for each item of the first :

```
words = ["one", "two"]
letters = [letter for word in words for letter in word]
print(letters)  # ['o', 'n', 'e', 't', 'w', 'o']
```

demo03
nested_comprehensions

SpiralTrain developer courses

Comprehension Features

- Unpack each tuple into names inside the `for` part.
- The expression can call a function.

```
def subtract(a: int, b: int) -> int:
    return a - b


pairs = [(6, 3), (1, 7), (5, 5)]
differences = [subtract(y, x) for x, y in pairs]
print(differences)  # [-3, 6, 0]
```

demo04
comprehension_features

Zip Function

- `zip` pairs items from its input iterables.
- By default it stops when the shortest input runs out.
- `strict=True` raises a `ValueError` when the inputs differ in length, instead of stopping early :

```
keys = ["a", "b", "c"]
values = [1, 2, 3]
pairs = list(zip(keys, values))
print(pairs)  # [('a', 1), ('b', 2), ('c', 3)]
checked = list(zip(keys, values, strict=True))  # the lengths match, so no error
print(checked)  # [('a', 1), ('b', 2), ('c', 3)]
```

demo05
zip_function

SpiralTrain developer courses

Dictionary Construction with zip

- Pass pairs of keys and values to `dict` to build a dictionary.

```
keys = ["a", "b", "c"]
values = [1, 2, 3]
counts = dict(zip(keys, values))
print(counts)  # {'a': 1, 'b': 2, 'c': 3}
```

demo06
dictionary_with_zip

SpiralTrain developer courses

Dictionary Comprehensions

- Use `key: value` to compute each dictionary entry.
- Add an `if` to keep only some entries.

```
squares = {number: number**2 for number in range(1, 6)}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
counts = {"a": 2, "b": 0, "c": 3}
positive = {key.upper(): value for key, value in counts.items() if value > 0}
print(positive)  # {'A': 2, 'C': 3}
```

demo07
dictionary_comprehension

SpiralTrain developer courses

Dictionary from Keys

- `dict.fromkeys` gives every key the same value.
- Use a comprehension when each key needs its own mutable value.

```
counts = dict.fromkeys(["a", "b", "c"], 0)
print(counts)  # {'a': 0, 'b': 0, 'c': 0}
groups: dict[str, list[int]] = {key: [] for key in ["a", "b", "c"]}
groups["a"].append(1)
print(groups)  # {'a': [1], 'b': [], 'c': []}
```

- Avoid `dict.fromkeys(keys, [])` : every key would share the same list.

demo08
dictionary_from_keys

SpiralTrain developer courses

Set Comprehensions

- A set comprehension uses braces around an expression.
- Duplicate results disappear; sets have no defined display order.

```
numbers = [1, 2, 2, 3, 4]
squares = {number**2 for number in numbers}
print(sorted(squares))  # [1, 4, 9, 16]
even_squares = {number**2 for number in numbers if number % 2 == 0}
print(sorted(even_squares))  # [4, 16]
```

demo09
set_comprehensions

SpiralTrain developer courses

Generator Expressions

- A generator expression supplies values as they are needed, without building a list.
- As the only argument of a call, it needs no extra parentheses.

```
numbers = [4, 8, 15, 16, 23, 42]
print(sum(n * n for n in numbers))  # 2854
print(any(n > 40 for n in numbers))  # True
print(all(n % 2 == 0 for n in numbers))  # False
print(any([]), all([]))  # False True
```

- `any` and `all` stop as soon as the answer is known.
- Generator functions, `yield` and generators in depth belong to Advanced Python.

demo10
generator_expressions

SpiralTrain developer courses

Built-ins that Take a Function

- `sorted` returns a new list; `list.sort` changes a list in place.
- A `key` function computes the comparison value for each item.
- `min` and `max` accept a `key` function too.

```
words = ["ticket", "to", "mars", "one", "way"]
print(sorted(words))  # ['mars', 'one', 'ticket', 'to', 'way']
print(sorted(words, key=len))  # ['to', 'one', 'way', 'mars', 'ticket']
print(min(words, key=len), max(words, key=len))  # to ticket
```

demo11
sorted_by_key

SpiralTrain developer courses

Recognising map and filter

- You will encounter these function-based forms in other people's code.
- `map` applies a function; `filter` keeps items for which a function returns true.
- Both return iterators; `list` collects their results.

```
def square(x: int) -> int:
    return x * x


def even(x: int) -> bool:
    return x % 2 == 0


print(list(map(square, range(1, 6))))  # [1, 4, 9, 16, 25]
print(list(filter(even, range(1, 6))))  # [2, 4]
print([square(x) for x in range(1, 6) if even(x)])  # [4, 16]
```

demo12
map_and_filter_functions

Comprehensions and map

- A comprehension often reads more directly than a combination of `map` and `filter`.
- Recognise both forms; choose the one that makes the operation clear.

```
mapped = list(map(lambda x: 2 * x + 1, [10, 20, 30]))
print(mapped)  # [21, 41, 61]
comprehended = [2 * x + 1 for x in [10, 20, 30]]
print(comprehended)  # [21, 41, 61]
```

SpiralTrain developer courses

Recognising reduce

- `functools.reduce` combines items into one result.
- Here each step multiplies the running result by the next number.

```
from functools import reduce

product = reduce(lambda x, y: x * y, [1, 2, 3, 4])
print(product)  # 24

result = 1
for number in [1, 2, 3, 4]:
    result *= number
print(result)  # 24
```

- Prefer a named built-in such as `sum` when it expresses the operation directly.

demo14
reduce_and_lambda

SpiralTrain developer courses

Summary : Comprehensions

- Build lists, dictionaries and sets with comprehensions.
- Use an optional `if` to filter the input.
- Keep nested comprehensions simple.
- Feed generator expressions into `sum`, `any` and `all`.
- Pass a `key` function to `sorted`, `min` and `max`.
- Recognise `map`, `filter` and `reduce` when reading other code.

exercise07
Comprehensions

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- ▶ **Exception Handling**
- Python IO
- Python Database Access
- Python Classes

APPENDIX

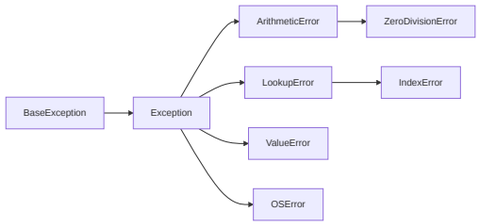
- Type Hinting
- Python Libraries

- Unexpected Errors
- Reading a Traceback
- Exception Handling
- Typed Exception Handling
- Exception Handling with Else
- except Clause Multiple Exceptions
- Python Standard Exceptions
- Assertions Check Programmer Assumptions
- try-finally Clause
- Example try-finally
- Exception Arguments
- Example Argument of an Exception
- Raising Exceptions
- Example Raising an Exception
- User Defined Exceptions

SpiralTrain developer courses

Unexpected Errors

- **Python provides exceptions to handle unexpected errors :**
  - Exceptions are events disrupting normal program flow
  - May occur during the execution of program instructions
- **When Python encounters conditions it can't cope with :**
  - An exception is raised
  - An exception is a Python object that represents an error
- **Exception types are classes, and those classes form a tree :**



- **Catching a class catches everything below it in that tree**
- **When a Python script raises an exception :**
  - Python looks for a matching handler in the callers; if none exists, the script stops
- **Another feature to handle unexpected errors are assertions :**
  - Assertions are tests which must be true to continue

SpiralTrain developer courses

Reading a Traceback

- Run `01_unexpectederrors.py`; it deliberately ends with an uncaught exception.

```
def divide(total: int, count: int) -> float:
    return total / count



divide(10, 0)
```

```
Traceback (most recent call last):
  File "01_unexpectederrors.py", line 5, in <module>
    divide(10, 0)
    ~~~~~~^^^^^^^
  File "01_unexpectederrors.py", line 2, in divide
    return total / count
           ~~~~~~^~~~~~~
ZeroDivisionError: division by zero
```

- Read bottom-up : the type and message, the failing line, then its caller.
- File paths are shortened here; line numbers point into the demo.

demo01
unexpectederrors

SpiralTrain developer courses

Exception Handling

- A `try` block contains code that may fail.
- A bare `except` catches everything, including keyboard interrupts.
- Recognise this form, but prefer specific exception types.

```
# A bare except is shown here so the next demo can improve it.
try:
    x = 1
    y = 0
    z = x / y
except:
    print("Cannot divide by zero")

try:
    fh = open("somefile", "r")
except:
    print("Cannot open file")
else:
    fh.close()
```

demo02
exceptionhandling

SpiralTrain developer courses

Typed Exception Handling

- Catch the errors you can handle.
- `FileNotFoundError` is a subclass of `OSError`.
- Put the specific handler first; Python uses the first matching handler.

```
try:
    fh = open("somefile", "r")
except FileNotFoundError:
    print("The file does not exist")
except OSError as error:
    print(f"Cannot open the file: {error}")
else:
    fh.close()
```

- Other exception types propagate to the caller.

demo03
typedexceptionhandling

SpiralTrain developer courses

Exception Handling with Else

- `else` runs only when the `try` block finishes without an exception.
- Keep code that does not need the handler in `else`.

```
try:
    x = int(input("Please enter a number: "))
except ValueError:
    print("Oops! That was not a valid number")
else:
    print(f"Yeah! {x} was a really good number")
```

demo04
exceptionhandlingwithelse

SpiralTrain developer courses

except Clause Multiple Exceptions

- Same `except` clause may also handle multiple exceptions :

```
try:
    Do your operations here;
except(Exception1[, Exception2[,...ExceptionN]]]):
    If there is any exception from the given exception list,
    then execute this block
else:
    If there is no exception then execute this block
```

- Example :

```
for value in (None, "Hi!"):
    try:
        print("Attempting to convert", value, "-->", end=" ")
        print(float(value))
    except (TypeError, ValueError):
        print("Something went wrong!")
```

- Disadvantage is error messages are less distinctive

demo05
multipleexceptions

Python Standard Exceptions

SpiralTrain developer courses

Exception	Typical cause
`ValueError`	A valid type has an unsuitable value, such as `int("six")`.
`TypeError`	An operation receives an unsuitable type.
`NameError`	A name has not been defined.
`AttributeError`	An object has no such attribute.
`IndexError`	A sequence index is out of range.
`KeyError`	A dictionary has no such key.
`ZeroDivisionError`	Division by zero.
`OSError`	A file or operating system operation fails.
`FileNotFoundError`	A requested file or directory does not exist.
`AssertionError`	An `assert` condition is false.
`EOFError`	`input()` reaches the end of the input, with Ctrl+D on Linux and macOS, or Ctrl+Z and Enter on Windows.
`KeyboardInterrupt`	The user presses Ctrl+C. It is a `BaseException`, so `except Exception` does not catch it.

SpiralTrain developer courses

Assertions Check Programmer Assumptions

- Use `assert` for an internal assumption, not for validating user input.
- Python can disable assertions with `-O`; raise an exception for required checks.

```
def kelvin_to_fahrenheit(temperature: float) -> float:
    assert temperature >= 0, "Colder than absolute zero!"
    return (temperature - 273.15) * 1.8 + 32


print(f"{kelvin_to_fahrenheit(273.15):.1f}")  # 32.0
print(kelvin_to_fahrenheit(-5))  # Deliberate AssertionError.
```

demo06
assertionerror

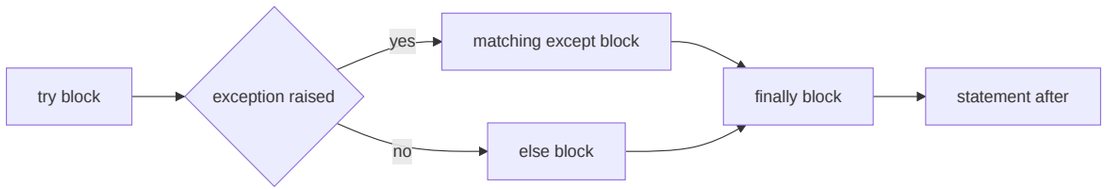
SpiralTrain developer courses

try-finally Clause

- `finally` block may be used along with a `try` block :
  - `finally` block is always executed
  - Whether or not the try-block raised an exception
- Syntax of the `try-finally` :

```
try:
    Do your operations here;
    Due to any exception, this may be skipped
finally:
    This would always be executed
```

- Combining `finally` and `except` is allowed :
  - `try` block can be followed by multiple `except` clauses and one `finally` clause



- `else` and `finally` can be combined :
  - `else` runs when no exception was raised, `finally` runs in every case

SpiralTrain developer courses

Example try-finally

- `finally` runs while an exception propagates to a handler.
- It does not handle the exception itself.

```
def divide(total: int, count: int) -> float:
    try:
        return total / count
    finally:
        print("This always runs")


try:
    divide(10, 0)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

```
This always runs
Cannot divide by zero
```

demo07
tryfinally

SpiralTrain developer courses

Exception Arguments

- `except ValueError as error` binds the exception object to `error`.
- `str(error)` gives its message; `print(error)` uses that text.
- `error.args` holds the constructor arguments as a tuple.
- Some types expose additional attributes, such as `OSError.filename`.

Example Argument of an Exception

```
def temp_convert(value: str) -> None:
    try:
        print(int(value))
    except ValueError as error:
        print("Argument does not contain numbers:", error)


temp_convert("xyz")
```

```
Argument does not contain numbers: invalid literal for int() with base 10: 'xyz'
```

demo08
exceptionarguments

Raising Exceptions

- **Exceptions can be raised with the `raise` statement :**

```
raise ExceptionType(args)
raise ExceptionType(args) from cause
raise
```

- **Explanation :**
  - `ExceptionType` is the class of the exception, for example `NameError`
  - `args` are passed to its constructor and kept in the exception's `args` attribute
  - `from cause` chains the new exception to the one that caused it
  - A bare `raise` inside an `except` block re-raises the exception being handled
- **Every exception is an instance of a class :**
  - Built-in exceptions derive from `BaseException`, nearly all of them from `Exception`
  - Your own exceptions should derive from `Exception` or one of its subclasses

Example Raising an Exception

```
try:
    raise NameError("HiThere")
```

- **In order to catch an exception :**
  - `except` should refer to same exception thrown as class object
- **To capture above exception, write except clause as follows :**

```
try:
    raise NameError("HiThere")
except NameError:
    print("An exception flew by!")
    raise  # re-raise it for a caller to handle
```

User Defined Exceptions

- **Created by deriving classes from standard built-in exceptions :**
  - Exception class may be created that is subclassed from `RuntimeError`
  - Useful when more specific information is needed when an exception is caught
- **In `try:` block :**
  - User-defined exception is raised and caught in the `except` block
  - The name `e` is bound to the `NetworkError` instance that was raised

```
class NetworkError(RuntimeError):
    def __init__(self, hostname: str) -> None:
        super().__init__(f"Bad hostname: {hostname}")
        self.hostname = hostname
```

- **With above class exception is raised as follows :**

```
try:
    raise NetworkError("example.invalid")
except NetworkError as e:
    print(e)  # Bad hostname: example.invalid
    print(e.hostname)  # example.invalid
```

demo10
userdefinedexceptions

SpiralTrain developer courses

Summary : Exception Handling

- **Exceptions are events disrupting normal program flow :**
  - Python provides exception handling mechanism for unexpected errors
- **Code that might raise an exception is placed in a `try` block :**
  - This code is tested for exceptions
- **`try` block is followed by an `except` block :**
  - Control jumps to this block in case of exceptions and exceptions are handled there
- **`try` block may also be followed by a `finally` block :**
  - Is always executed whether or not the try-block raised an exception
- **Exceptions can have an argument :**
  - Value that gives additional information about the problem
- **Python has a number of predefined exceptions like :**
  - `ValueError`, `TypeError`, `KeyError` and `OSError`
- **Exceptions can be raised with the `raise` statement**
- **It is possible to create User Defined Exceptions :**
  - By deriving classes from standard built-in exceptions

exercise08
Exception Handling

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- ▶ **Python IO**
- Python Database Access
- Python Classes

APPENDIX

- Type Hinting
- Python Libraries

- Input and Output
- IO Module
- Opening Files
- File Open Modes
- Result of Calling open
- File Object Attributes
- Bytes and Byte Arrays
- Reading Binary Files
- Writing Binary Files
- Writing Text Files
- Reading Text Files
- Closing Files
- Using with
- File Positions
- The csv Module
- Reading CSV Rows by Name
- Paths with pathlib
- Paths Relative to a Module
- Renaming and Deleting Files
- Creating and Removing Directories
- Directory Contents

SpiralTrain developer courses

Input and Output

- `print` writes its arguments to standard output.
- `input` reads one line from standard input and removes the trailing newline.
- It returns a string; convert it before doing arithmetic.

```
print("Python is really a great language,", "isn't it?")
text = input("Enter some input : ")
print(text)
```

demo01
readinput

IO Module

- Core of the I/O system is implemented in io library module
- Consists of a collection of different I/O classes :
  - `FileIO`
  - `BufferedReader`
  - `BufferedWriter`
  - `BufferedRWPair`
  - `BufferedRandom`
  - `TextIOWrapper`
  - `BytesIO`
  - `StringIO`
- Each class implements a different kind of I/O
- Classes get layered to add features

SpiralTrain developer courses

Opening Files

- `open` function :
  - Opens a file to be able to read or write a file later
  - Creates file object which can be used to call other support methods

```
file_object = open(file, mode="r", buffering=-1, encoding=None)
```

- `file` :
  - String value that contains the name of the file to be accessed
- `mode` :
  - Determines mode in which the file has to be opened, read, write, append etc.
  - `t` for text mode is default and optional, `b` is for byte mode
- `buffering` :
  - `0` disables buffering, but is allowed only in binary mode
  - `1` enables line buffering in text mode
  - If buffering > 1 : Buffering performed with indicated buffer size

File Open Modes

- **Python distinguishes between binary and text I/O :**
  - In binary mode content is returned as bytes objects without any decoding
  - In text mode, as `str`, bytes decoded using given or platform-specific encoding

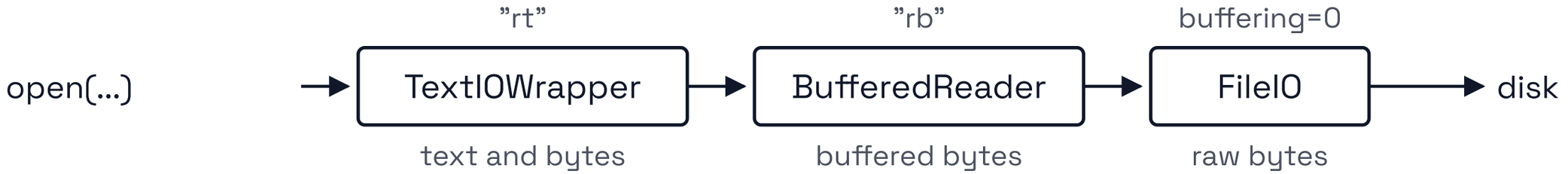
Mode	Description
`r` / `rb`	Reading only, text or binary format. Default is text mode
`r+` / `rb+`	Both reading and writing, text or binary format
`w` / `wb`	Writing only, text or binary. Overwrites existing file or creates new file
`w+` / `wb+`	Both writing and reading, text or binary. Overwrites or creates file
`a` / `ab`	Appending, text or binary. File pointer is at the end of the file
`a+` / `ab+`	Both appending and reading, text or binary format

Result of Calling open

- The object returned varies with the file mode and the buffering :

```
>>> open("test.txt", "rt", encoding="utf-8")
<_io.TextIOWrapper name='test.txt' mode='rt' encoding='utf-8'>
>>> open("test.txt", "rb")
<_io.BufferedReader name='test.txt'>
>>> open("test.txt", "rb", buffering=0)
<_io.FileIO name='test.txt' mode='rb'>
```

- Opening a text file stacks three objects, and you are handed the outermost :



demo02
openfile

SpiralTrain developer courses

File Object Attributes

- File objects expose their name, mode and closed state.

```
fo = open("test_02.txt", "wb")
print("Name of the file:", fo.name)
print("Closed or not :", fo.closed)
print("Opening mode :", fo.mode)
fo.close()
```

```
Name of the file: test_02.txt
Closed or not : False
Opening mode : wb
```

demo03
fileioobject

SpiralTrain developer courses

Bytes and Byte Arrays

- `bytes` is immutable; `bytearray` allows individual bytes to change.
- Indexing returns an integer from 0 to 255, not a character.
- Encoding converts text to bytes; decoding converts bytes back to text.

```
text = "ABCDE"
arr = bytes(text, "utf-8")
print(arr)  # b'ABCDE'
print(arr[0])  # 65
print(arr.decode("utf-8"))  # ABCDE

mutable = bytearray(arr)
mutable[0] = 70
print(mutable)  # bytearray(b'FBCDE')
print(mutable.decode("utf-8"))  # FBCDE
```

- Binary files read and write bytes; text files handle encoding for you.

demo04
bytesandbytesarrays

SpiralTrain developer courses

Reading Binary Files

- Binary mode (`rb`) reads bytes without decoding them.
- `read(3)` reads up to three bytes; `read()` reads the remaining contents.
- Binary mode takes no `encoding` argument.

```
fr = open("trilobyte.jpg", mode="rb")
data = fr.read(3)
print(type(data))  # <class 'bytes'>
print(data.hex())  # ffd8ff
fr.close()
```

demo05
readbinaryfile

Writing Binary Files

- `wb` writes bytes and replaces any existing file.
- `struct` packs numbers into bytes and unpacks them with the same format.
- Here `<iii` means three four-byte integers in little-endian order.

```
from struct import pack, unpack

payload = pack("<iii", 1, 2, 3)
fw = open("numbers.bin", mode="wb")
fw.write(payload)
fw.close()

fr = open("numbers.bin", mode="rb")
print(unpack("<iii", fr.read()))  # (1, 2, 3)
fr.close()
```

demo06
writebinaryfile

SpiralTrain developer courses

Writing Text Files

- **Open file for writing specifying mode and encoding :**

```
fw = open("textfilepolish.txt", mode="w", encoding="UTF-8")
```

- **`write` method writes any string to an open file :**
  - Parameter is the content to be written into the opened file
  - Does not add a newline character ('\n') to the end of the string

```
s = "Polish text: ąćęłńóśźżĄĆĘŁŃÓŚŹŻ"
fw.write(s)
```

- **Always close the file when you are done :**

```
fw.close()
```

- **File `textfilepolish.txt` with following content is created :**

```
Polish text: ąćęłńóśźżĄĆĘŁŃÓŚŹŻ
```

demo07
writetextfile

SpiralTrain developer courses

Reading Text Files

- **Open file for reading specifying mode and encoding :**

```
fr = open("textfilepolish.txt", mode="r", encoding="UTF-8")
```

- **`read` method reads a string from an open file :**
  - In text mode the count is a number of characters, not bytes

```
file_object.read([count])
```

- **`read()` starts at the current file position :**
  - If count is missing then it tries to read as much as possible, may be until end of file
- **Example of reading and closing file `textfilepolish.txt` :**

```
data = fr.read()
print(data)
fr.close()
```

- **Result :**

```
Polish text: ąćęłńóśźżĄĆĘŁŃÓŚŹŻ
```

Closing Files

- `close` method flushes unwritten data and closes file object :
  - No more writing can be done after calling `close`
  - Do not rely on garbage collection to close a file promptly
- Syntax :

```
file_object.close()
```

- Example :

```
fo = open("foo.txt", "wb")
print("Name of the file:", fo.name)
fo.close()
```

- Result :

```
Name of the file: foo.txt
```

- Good practice to use `close()` method to close a file, but see the next slide

Using with

- `with` closes the file when the block ends, even after an exception or `return`.
- Prefer this form whenever you open a file.

```
with open("foo.txt", mode="w", encoding="utf-8") as fw:
    fw.write("one line\n")
# The file is closed here, even if write() raised.
print(fw.closed)  # True

with open("foo.txt", encoding="utf-8") as fr:
    print(fr.read(), end="")  # one line
```

demo08
usingwith

File Positions

- `tell()` reports the current position; `seek(0)` returns to the start.
- Binary file positions count bytes from the beginning.
- Text file positions are opaque values; use `seek(0)` or a value from `tell()`.

```
with open("positions.bin", "wb") as fw:
    fw.write(b"ABCDE")

with open("positions.bin", "rb") as fr:
    print(fr.read(2))  # b'AB'
    print(fr.tell())  # 2
    fr.seek(0)
    print(fr.read(2))  # b'AB'
```

demo09
file_positions

The csv Module

- Use `csv` for tabular text; it handles quoted commas and embedded newlines.
- Open CSV files with `newline=""` so `csv` controls newline handling.

```
import csv

with open("planets.csv", "w", newline="", encoding="utf-8") as fw:
    writer = csv.writer(fw)
    writer.writerow(["name", "moons"])
    writer.writerows([["Earth", 1], ["Mars", 2]])

with open("planets.csv", newline="", encoding="utf-8") as fr:
    for row in csv.reader(fr):
        print(row)
```

demo10
csv_module

SpiralTrain developer courses

Reading CSV Rows by Name

- `DictReader` uses the first row as column names.
- Field values are strings; convert numeric fields explicitly.
- The demo creates the same CSV file, then reads it by column name :

```
with open("planets.csv", newline="", encoding="utf-8") as fr:
    for row in csv.DictReader(fr):
        name = row["name"]
        moons = int(row["moons"])
        print(f"{name}: {moons} moons")
```

```
Earth: 1 moons
Mars: 2 moons
```

- `csv.reader` returns lists instead : `['Earth', '1']`.

demo11
csv_dictionary

SpiralTrain developer courses

Paths with pathlib

- **The `pathlib` module represents a path as an object, not a string :**
  - `Path` knows how to join, inspect, read, write, rename and delete
  - It is the modern replacement for stringly typed `os.path` work

```
from pathlib import Path

here = Path.cwd()                    # the current working directory
sample = here / "data" / "one.txt"   # joining, with / instead of concatenation
```

- **The `/` operator joins path segments :**
  - It produces the right separator on Windows, macOS and Linux alike
  - No more `"dir" + "\\" + name`, and no more forgetting the separator
- **A path answers questions about itself :**

```
sample.name      # one.txt
sample.stem      # one
sample.suffix    # .txt
sample.parent    # the containing directory, as a Path
sample.exists()  # True or False
```

Paths Relative to a Module

- `Path(__file__)` is the path of the running module, so `parent` is its folder :
  - `resolve()` makes the path absolute, and follows any symbolic links
- `path.open()` works like `open()`, and takes the same arguments :

```
from pathlib import Path

here = Path(__file__).resolve().parent  # the folder of this module
print(here.is_absolute())  # True

with Path(__file__).open(encoding="utf-8") as source:
    print(source.readline().strip())  # from pathlib import Path
```

SpiralTrain developer courses

Renaming and Deleting Files

- A `Path` renames and deletes itself :

```
from pathlib import Path

data_file = Path("datafile1.txt")
data_file.touch()
renamed = data_file.rename("renamedfile.txt")
renamed.unlink()
```

- `touch` creates an empty file, `unlink` removes one :
  - `unlink(missing_ok=True)` does nothing when the file is already gone
- Short reads and writes need no `open` at all :

```
sample = Path("sample.txt")
sample.write_text("first\n", encoding="utf-8")
content = sample.read_text(encoding="utf-8")
```

- The `os` module still has `os.rename` and `os.remove` :
  - They take strings and are what older code uses

demo12
renaming_files

SpiralTrain developer courses

Creating and Removing Directories

- `mkdir` creates a directory, `rmdir` removes an empty one :

```
from pathlib import Path

Path("test1").mkdir(exist_ok=True)
Path("test1").rmdir()
```

- Two keyword arguments cover the awkward cases :
  - `parents=True` creates every missing directory in the path
  - `exist_ok=True` succeeds quietly when the directory is already there

```
Path("test2/nested/deeper").mkdir(parents=True, exist_ok=True)
```

- `rmdir` only removes an empty directory :
  - Remove the contents first, from the leaf upwards
  - `shutil.rmtree` removes a whole tree at once, and does not ask twice

demo13
creatingandremoving_directories

SpiralTrain developer courses

Directory Contents

- `iterdir` lists direct entries; `glob` selects matching names.
- Each entry is a `Path` that you can inspect or open.
- `rglob` searches subdirectories too.

```
from pathlib import Path

here = Path.cwd()
work = here / "justtesting"
work.mkdir(exist_ok=True)
(work / "one.txt").write_text("first\n", encoding="utf-8")
(work / "two.txt").write_text("second\n", encoding="utf-8")
print("contents          :", sorted(p.name for p in work.iterdir()))
print("only the text     :", sorted(p.name for p in work.glob("*.txt")))
```

demo14
directorymethods

SpiralTrain developer courses

Summary : Python IO

- **Simplest way to produce output is to use the `print` function :**
  - Allows you to pass zero or more expressions, separated by commas
- **The `input` function reads a line of text from standard input :**
  - It always returns a string, so convert it before doing arithmetic with it
- **Python also has File Object methods to manipulate files :**
  - Creating, opening files, closing files, reading, writing, renaming and deleting files
- **Files can be opened in several modes :**
  - For reading, for writing, for appending or combinations thereof
- **Binary files contain bytes; text files encode and decode strings.**
- **Use `csv.reader`, `csv.writer` and `DictReader` for tabular text.**
- **Open files with `with`, which closes them whatever happens :**
  - The manual `open` and `close` pair is shown so that you recognise older code
- **`pathlib` represents a path as an object :**
  - `Path` joins with `/`, and creates, renames, deletes, reads and lists

exercise09
Python IO

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- **Python Database Access**
- Python Classes

APPENDIX

- Type Hinting
- Python Libraries

- Python DB API
- DB API Concepts
- Check Database Version
- Using with
- Inserting Data
- Prepared Statements
- Last inserted row id
- Retrieving Data
- Filtering and Grouping Data
- Rows by Column Name
- Fetching Rows
- Parameterized Queries
- Metadata
- Transactions
- Committing Data

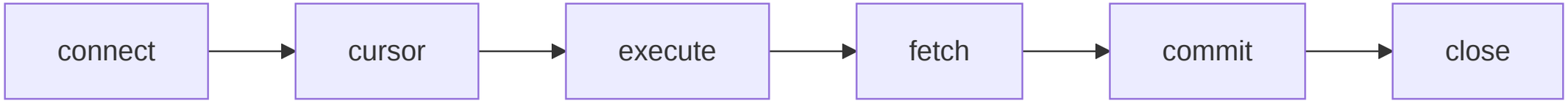
SpiralTrain developer courses

Python DB API

- **Provides a standard interface from Python to databases :**
  - Most Python database interfaces adhere to this standard
  - Each database has its own module highly consistent with others
- **The standard is described in PEP 249 :**
  - Modules are available for most of the popular relational databases
  - Code written for one database reads almost the same for another
- **Python DB API 2.0 compatible modules :**
  - `sqlite3` (SQLite), part of the standard library and used in this module
  - `psycopg` (PostgreSQL)
  - `mysqlclient` or `PyMySQL` (MySQL and MariaDB)
  - `oracledb` (Oracle)
  - `pyodbc` (any database with an ODBC driver, such as SQL Server and DB2)

DB API Concepts

- **DB API uses two main concepts for processing queries :**
  - Connections and cursors
- **Every script follows the same shape :**



- **Connection objects logically wrap a database connection :**
  - Provide access to the database and handle transactions
- **`sqlite3.connect()` opens a database file, and creates it if it does not exist**
- **Cursor objects used to manipulate and query the database :**
  - Created on connections using the `conn.cursor()` method
  - Cursor objects provide ways to execute statements and access results

```
con = sqlite3.connect("sample.db")
cur = con.cursor()
cur.execute("CREATE TABLE testtable(id int, name varchar(254))")
```

Check Database Version

```
import sqlite3
import sys
con = None
try:
    con = sqlite3.connect("test.db")  # creates test.db if it does not exist
    cur = con.cursor()
    cur.execute("SELECT SQLITE_VERSION()")
    data = cur.fetchone()
    print(f"SQLite version: {data}")
except sqlite3.Error as e:
    print(f"Error: {e}")
    sys.exit(1)
finally:
    if con:
        con.close()
```

demo01
databaseversion

SpiralTrain developer courses

Using with

- **A connection used in a `with` statement manages the transaction :**
  - When the block ends normally, the changes are committed
  - When the block ends with an exception, the changes are rolled back

```
import sqlite3
con = sqlite3.connect("test.db")
with con:
    cur = con.cursor()
    cur.execute("SELECT SQLITE_VERSION()")
    data = cur.fetchone()
    print(f"SQLite version: {data}")
con.close()  # `with con:` commits or rolls back, it does not close
```

- **The connection stays open after the block :**
  - Close it yourself with `con.close()` when you are done
- **`closing()` from `contextlib` calls `close()` when the block ends, so one `with` line can commit and close :**

```
with closing(sqlite3.connect("test.db")) as con, con:
    cur = con.cursor()
    cur.execute("SELECT SQLITE_VERSION()")
    print(f"SQLite version: {cur.fetchone()}")
```

demo02
usingwith

SpiralTrain developer courses

Inserting Data

- **Cursor objects provides a `cursor.execute()` method :**
  - Execute SQL statements on a connection
- **Method also accepts :**
  - Sequence of parameters to customize the SQL statement
- **This demo creates the `Cars` table that the later demos read :**

```
import sqlite3
con = sqlite3.connect("cars.db")
with con:
    cur = con.cursor()
    cur.execute("DROP TABLE IF EXISTS Cars")
    cur.execute("CREATE TABLE Cars(Id INT, Name TEXT, Price INT)")
    cur.execute("INSERT INTO Cars VALUES(1,'Audi',52642)")
    cur.execute("INSERT INTO Cars VALUES(2,'Mercedes',57127)")
    cur.execute("INSERT INTO Cars VALUES(3,'Skoda',9000)")
    cur.execute("INSERT INTO Cars VALUES(4,'Volvo',29000)")
    cur.execute("INSERT INTO Cars VALUES(5,'Bentley',350000)")
    cur.execute("INSERT INTO Cars VALUES(6,'Citroen',21000)")
    cur.execute("INSERT INTO Cars VALUES(7,'Hummer',41400)")
    cur.execute("INSERT INTO Cars VALUES(8,'Volkswagen',21600)")
```

demo03
insertingdata

SpiralTrain developer courses

Prepared Statements

- **If same query must be repeated multiple times :**
  - Can use parameterized queries with question marks or named parameters
  - Good for both performance and for security
- **Can use `executemany()` method for prepared statements :**

```
import sqlite3
cars = (
    (1, "Audi", 52642),
    (2, "Mercedes", 57127),
    (3, "Skoda", 9000),
    (4, "Volvo", 29000),
    (5, "Bentley", 350000),
    (6, "Citroen", 21000),
    (7, "Hummer", 41400),
    (8, "Volkswagen", 21600),
)
con = sqlite3.connect("cars.db")
with con:
    cur = con.cursor()
    cur.execute("DROP TABLE IF EXISTS Cars")
    cur.execute("CREATE TABLE Cars(Id INT, Name TEXT, Price INT)")
    cur.executemany("INSERT INTO Cars VALUES(?, ?, ?)", cars)
```

demo04
preparedstatements

SpiralTrain developer courses

Last inserted row id

- **Sometimes we need to determine the id of last inserted row :**
  - In Python SQLite we use the `lastrowid` attribute of the cursor object
- **The name `":memory:"` opens a database that lives only while the program runs :**

```
import sqlite3
con = sqlite3.connect(":memory:")
with con:
    cur = con.cursor()
    cur.execute("CREATE TABLE Friends(Id INTEGER PRIMARY KEY, Name TEXT)")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Tom')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Rebecca')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Jim')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Robert')")
    lid = cur.lastrowid
    print(f"The last Id of the inserted row is {lid}")
```

demo05
lastinsertedrowid

Retrieving Data

- `fetchall()` returns all rows of the result as a list of tuples :
  - The `Cars` table comes from demo03 or demo04, so run one of them first

```
import sqlite3
con = sqlite3.connect("cars.db")
with con:
    cur = con.cursor()
    cur.execute("SELECT * FROM Cars")
    rows = cur.fetchall()
    for row in rows:
        print(row)
```

```
(1, 'Audi', 52642)
(2, 'Mercedes', 57127)
(3, 'Skoda', 9000)
```

- Other fetch methods read part of the result :

```
first_row = cur.fetchone()
next_10_rows = cur.fetchmany(10)
all_remaining_rows = cur.fetchall()
```

Filtering and Grouping Data

- `WHERE` filters rows; `?` binds the value from the parameter tuple
- `GROUP BY` forms groups; `COUNT(*)` counts rows and `SUM` adds non-NULL values
- `COALESCE(value, 0)` replaces SQL `NULL` with zero, including an empty `SUM`
- `ORDER BY` sorts results; `DESC` puts the largest total first

```
import sqlite3

# Run demo03 or demo04 first to create the Cars table.
con = sqlite3.connect("cars.db")
try:
    cur = con.cursor()
    cur.execute("""
        SELECT Name, COUNT(*), COALESCE(SUM(Price), 0)
        FROM Cars WHERE Price >= ?
        GROUP BY Name ORDER BY SUM(Price) DESC
        """, (50000,))
    for row in cur.fetchall():
        print(row)
finally:
    con.close()
```

demo07
aggregatingdata

SpiralTrain developer courses

Rows by Column Name

- **A row is a tuple by default, so a column is read by its position :**
  - `row[1]` works, but the reader has to know what the second column is
- **Set `row_factory` on the connection to `sqlite3.Row` :**
  - Every row can then be read by column name as well as by position

```
import sqlite3
con = sqlite3.connect("cars.db")
con.row_factory = sqlite3.Row  # rows can now be read by column name
with con:
    cur = con.cursor()
    cur.execute("SELECT * FROM Cars")
    rows = cur.fetchall()
    for row in rows:
        print(f"{row['Id']} {row['Name']} {row['Price']}")
```

```
1 Audi 52642
2 Mercedes 57127
```

demo08
rowfactory

Fetching Rows

- **Returning all data at a time may not be feasible :**
  - It is also possible to fetch rows one by one with `fetchone()` :

```
import sqlite3
con = sqlite3.connect("cars.db")
with con:
    cur = con.cursor()
    cur.execute("SELECT * FROM Cars")
    while True:
        row = cur.fetchone()
        if row is None:
            break
        print(row[0], row[1], row[2])
```

- **`fetchone()` method returns next row from the table :**
  - If there is no more data left, it returns `None`

demo09
fetchone

SpiralTrain developer courses

Parameterized Queries

- **Passing data from Python to the database :**
  - First discouraged option is quoting values and sending plain SQL to the database
  - Second option preferred using binding parameters in the SQL with place holders
- **Python sqlite3 module supports two types of placeholders :**
  - Question marks and named placeholders

```
import sqlite3
car_id = 1
new_price = 62300
con = sqlite3.connect("cars.db")
with con:
    cur = con.cursor()
    cur.execute("UPDATE Cars SET Price=? WHERE Id=?", (new_price, car_id))
    print(f"Number of rows updated: {cur.rowcount}")
```

demo10
parameterizedqueries

Metadata

- **Metadata is information about the data in the database :**
  - Contains information about the tables and columns like name and datatypes

```
import sqlite3
con = sqlite3.connect("cars.db")
with con:
    cur = con.cursor()
    cur.execute("PRAGMA table_info(Cars)")
    data = cur.fetchall()
    for d in data:
        print(d[0], d[1], d[2])
```

```
0 Id INT
1 Name TEXT
2 Price INT
```

- **Metadata in SQLite can be obtained using `PRAGMA` command :**
  - SQLite objects may have attributes, which are metadata
  - Obtain specific metadata from querying the SQLite system `sqlite_master` table

demo11
metadata

SpiralTrain developer courses

Transactions

- **Transactions logically wrap statements into blocks of execution :**
  - Transactions are units of recovery
- **Main benefit is that you can undo changes very easily :**
  - Ensures data consistency at all times
  - Not all databases provide transactions !
- **Transactions conform to ACID principles :**
  - Atomicity, Consistency, Isolation, Durability
- **DB API supports transactions if the database supports them :**
  - Via methods on the connection object
- **Write all changes of the last transaction block to the database :**

```
con.commit()
```

- **Undo all changes applied to the database on the connection :**

```
con.rollback()
```

Committing Data

```
import sqlite3
import sys
con = None
try:
    con = sqlite3.connect("friends.db")
    cur = con.cursor()
    cur.execute("DROP TABLE IF EXISTS Friends")
    cur.execute("CREATE TABLE Friends(Id INTEGER PRIMARY KEY, Name TEXT)")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Tom')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Rebecca')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Jim')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Robert')")
    con.commit()  # without this line, close() discards the inserts
except sqlite3.Error as e:
    if con:
        con.rollback()
    print(f"Error: {e}")
    sys.exit(1)
finally:
    if con:
        con.close()
```

SpiralTrain developer courses

Summary : Python Database Access

- **Python DB API is standard interface from Python to databases :**
  - Each database has its own module highly consistent with others
- **DB API uses two main concepts for processing queries :**
  - Connections and cursors
- **Connection objects logically wrap a database connection :**
  - Provide means to handle database transactions
- **Cursor objects used to manipulate and query the database :**
  - Created on connections using the `conn.cursor()` method
- **If same query must be repeated multiple times :**
  - Can use parameterized queries with question marks or named parameters
- **A connection in a `with` statement commits or rolls back the transaction :**
  - It does not close the connection; `con.close()` does that
- **DB API supports transactions if database supports them :**
  - Via methods on the connection object

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- ▶ **Python Classes**

APPENDIX

- Type Hinting
- Python Libraries

- Object Orientation in Python
- Example Class and Objects
- Creating Classes
- Example Creating Classes and Objects
- Class Members
- Creating and Using Objects
- Accessing Attributes
- Class Attributes and Instance Attributes
- `__str__` and `__repr__`
- Attributes Are Public
- A Computed Attribute
- When a Plain Attribute Is Not Enough
- The Rule of Thumb
- Internal Attributes
- A Class Can Extend Another

SpiralTrain developer courses

Object Orientation in Python

- **Python is an object-oriented language :**
  - Every value in Python is an object, and every object has a class
  - Classes of your own are created with the `class` keyword
- **Class :**
  - A blueprint for a set of related objects
  - Defines the attributes and methods that every object of the class has
  - Attributes hold data, methods are functions; both are reached with dot notation
- **Object or instance :**
  - An individual object of a certain class, with its own identity
  - Its state is the current value of its attributes
  - State differs per object, behavior is shared by all instances
- **Inheritance :**
  - A class can extend another class, inheriting its attributes and methods

SpiralTrain developer courses

Example Class and Objects

- A class is a blueprint; each object has its own state :

	Car Class	Object Car A	Object Car B
Instance Variables	Plate Number	ABC 111	XYX 123
	Color	Blue	Red
	Manufacturer	Mitsubishi	Toyota
	Current Speed	50 km/h	100 km/h
Instance Methods	Accelerate Method		
	Turn Method		
	Brake Method		

SpiralTrain developer courses

Creating Classes

- **The `class` statement creates a new class :**
  - The name of the class follows the keyword `class`, then a colon
  - The indented block below it is the body of the class

```
class Person:
    """A person, with nothing in it yet."""

    pass  # an empty block
```

- **The string on the first line of the body is the documentation string :**
  - It can be read as `Person.__doc__`
- **Calling the class creates an object :**

```
p1 = Person()
print(p1)
```

```
<__main__.Person object at 0x000001CECE9D8D70>
```

demo01
simplestclass

SpiralTrain developer courses

Example Creating Classes and Objects

```python
class Employee:
    """Common base class for all employees."""
    emp_count = 0
    def __init__(self, name: str, salary: float) -> None:
        self.name = name
        self.salary = salary
        Employee.emp_count += 1
    def display_count(self) -> None:
        print(f"Total Employee {Employee.emp_count}")
    def display_employee(self) -> None:
        print(f"Name : {self.name}, Salary: {self.salary}")
```

- **To create an instance of a class :**
  - Call the class by its name and pass the arguments that `__init__` accepts
- **Create an object of the `Employee` class :**

```python
emp1 = Employee("Albert", 2000)
```

SpiralTrain developer courses

Class Members

- `__init__()` :
  - The initialization method, often called the constructor
  - Called by Python when you create a new instance of this class
  - Sets the instance attributes, here `self.name` and `self.salary`
- **Instance methods :**
  - Declared like normal functions, but the first parameter of each method is `self`
  - You do not pass `self` when you call a method, since Python adds it
- **`self` is a reference to the instance :**
  - Instance attributes are read and written as `self.name`
  - `self` is visible and explicit in every method definition
- **`emp_count` :**
  - A class attribute, shared by all instances, read as `Employee.emp_count`
- **Python frees objects for you when nothing refers to them any more :**
  - `__del__` exists and you rarely need it

SpiralTrain developer courses

Creating and Using Objects

- Creating objects :

```
# This would create first object of Employee class
emp1 = Employee("Albert", 2000)
# This would create second object of Employee class
emp2 = Employee("Werner", 5000)
```

- Access the object's methods and attributes using the dot operator :

```
emp1.display_employee()
emp2.display_employee()
print(f"Total Employee {Employee.emp_count}")
```

- Result :

```
Name : Albert, Salary: 2000
Name : Werner, Salary: 5000
Total Employee 2
```

demo02
employee

SpiralTrain
developer courses

Accessing Attributes

- In Python you can add, remove, or modify attributes at any time :

```
emp1.age = 7  # Add an 'age' attribute
emp1.age = 8  # Modify 'age' attribute
del emp1.age  # Delete 'age' attribute
```

- Four functions do the same with an attribute name held in a string :
  - Useful when the name is only known while the program runs

```
attr = "age"  # an attribute name held in a variable
print(hasattr(emp1, attr))  # True if the attribute exists
setattr(emp1, attr, 8)  # Set or create the attribute
print(getattr(emp1, attr))  # Returns the value of the attribute
delattr(emp1, attr)  # Delete the attribute
print(getattr(emp1, attr, "no age"))  # The default, because the attribute is gone
```

```
False
8
no age
```

Class Attributes and Instance Attributes

```python
class Car:
    wheels = 4  # class attribute, one value shared by every car
    def __init__(self, plate: str) -> None:
        self.plate = plate  # instance attribute, one value per car

a = Car("ABC 111")
b = Car("XYX 123")
print(a.plate, b.plate)  # ABC 111 XYX 123
print(a.wheels, b.wheels)  # 4 4
Car.wheels = 6  # changes the shared value
print(a.wheels, b.wheels)  # 6 6
a.wheels = 3  # creates an instance attribute on a only
print(a.wheels, b.wheels, Car.wheels)  # 3 6 6
```

- A class attribute is assigned in the class body; an instance attribute on `self`
- Reading `a.wheels` looks on the object first, then on the class
- Assigning `a.wheels` always writes to the object, so change shared data on the class
- Methods that need no instance exist as well, `@staticmethod` and `@classmethod`:
  - They are covered in the Advanced Python course

demo03
classattributes

SpiralTrain developer courses

`__str__` and `__repr__`

```
class Car:
    def __init__(self, plate: str, color: str) -> None:
        self.plate = plate
        self.color = color
    def __repr__(self) -> str:
        return f"Car({self.plate!r}, {self.color!r})"
    def __str__(self) -> str:
        return f"{self.color} car {self.plate}"

car = Car("ABC 111", "Blue")
print(car)  # str(), for people
print(repr(car))  # repr(), for developers
print([car])  # a list shows the repr() of its items
```

```
Blue car ABC 111
Car('ABC 111', 'Blue')
[Car('ABC 111', 'Blue')]
```

- `__str__` is used by `print()`, `str()` and f-strings
- `__repr__` is used by the interactive prompt, by containers and by `!r`:
  - Make it look like the call that creates the object; `str()` falls back to it

demo04
strrepr

SpiralTrain developer courses

Attributes Are Public

- **An attribute set in `__init__` is public, and that is fine :**
  - Any code that has the object can read it and assign to it
  - No methods are written to get or set it

```
class Square:
    def __init__(self, length: float) -> None:
        self.length = length

sq = Square(5)
print(sq.length)  # read it
sq.length = 6  # write it
print(sq.length)
```

```
5
6
```

demo05
publicattributes

SpiralTrain developer courses

A Computed Attribute

- **The area of a square depends on its length :**
  - As a method, `def area(self)`, it is called as `sq.area()`
  - With `@property` above the `def`, it is read as `sq.area`, like an attribute

```
class Square:
    def __init__(self, length: float) -> None:
        self.length = length
    @property
    def area(self) -> float:
        return self.length * self.length

sq = Square(5)
print(sq.area)  # no parentheses
sq.length = 6
print(sq.area)  # always current
sq.area = 50  # read-only : there is no setter
```

```
25
36
AttributeError: property 'area' of 'Square' object has no setter
```

demo06
computedproperty

SpiralTrain developer courses

When a Plain Attribute Is Not Enough

- The `Square` so far accepts any length, `sq.length = -3` included
- The fix makes `length` a property whose setter checks the value :

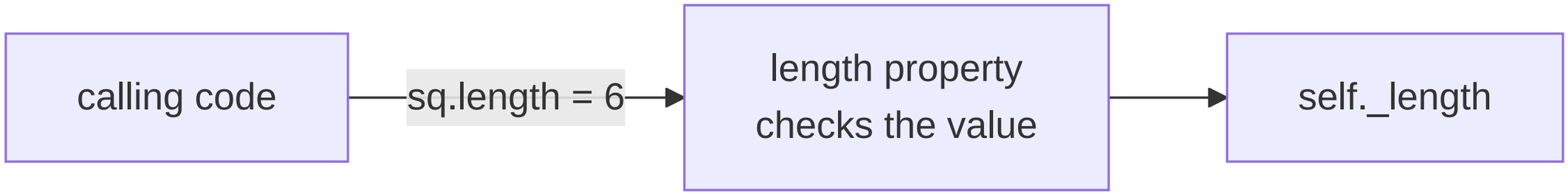
```python
class Square:
    def __init__(self, length: float) -> None:
        self.length = length  # goes through the setter below
    @property
    def length(self) -> float:
        return self._length
    @length.setter
    def length(self, value: float) -> None:
        if value < 0:
            raise ValueError(f"length must not be negative, got {value}")
        self._length = value
```

- The value is stored in `self._length`, the property's own storage
- The code that uses the class does not change :
  - `sq.length = 6` still works, and `sq.length = -3` now raises `ValueError`

demo07
validatingsetter

The Rule of Thumb

- **Start with a plain attribute :**
  - Do not write getter and setter methods in advance
- **Turn it into a property the day it needs a rule :**
  - A check on the value, or a value computed from other attributes
  - The calling code keeps writing `sq.length`, so nothing else changes
- **The property keeps its value in an attribute with a leading underscore :**
  - `_length` is internal to the class, by convention



SpiralTrain developer courses

Internal Attributes

- **Python has no keyword that makes an attribute private :**
  - A leading underscore, `_balance`, means internal, by convention
  - Code outside the class can reach it, and is expected not to
- **Two leading underscores, `__pin`, make Python rename the attribute to `_Account__pin`**

```
class Account:
    def __init__(self, owner: str) -> None:
        self.owner = owner  # public
        self._balance = 0  # internal, by convention
        self.__pin = "1234"  # name mangled to _Account__pin

acc = Account("Albert")
print(acc.owner)
print(acc._balance)  # works, but the underscore says: not for you
print(acc._Account__pin)  # the mangled name still works
```

```
Albert
0
1234
```

demo08
internalattributes

SpiralTrain developer courses

A Class Can Extend Another

```python
class Employee:
    def __init__(self, name: str, salary: float) -> None:
        self.name = name
        self.salary = salary
    def describe(self) -> str:
        return f"{self.name} earns {self.salary}"

class Manager(Employee):
    def __init__(self, name: str, salary: float, team: list[str]) -> None:
        super().__init__(name, salary)  # the Employee part
        self.team = team  # what a Manager adds
    def describe(self) -> str:  # replaces Employee.describe
        return f"{super().describe()} and leads {len(self.team)} people"
```

- `Manager` inherits every method of `Employee`, adds to it and may replace methods
- `super().__init__(...)` runs the `Employee` initialization, so `name` and `salary` are set
- `Manager("Marie", 6000, ["Albert", "Werner"]).describe()` returns:
  - `Marie earns 6000 and leads 2 people`
- Inheritance in depth is covered in the Advanced Python course

demo09
inheritance

SpiralTrain developer courses

Summary : Python Classes

- **Classes are blueprints for a set of related objects :**
  - Define both attributes and methods and are created with the `class` keyword
- **Objects are instances of classes :**
  - Each object has its own state and identity, instance methods have a `self` parameter
- **The initialization method is named `__init__()` :**
  - Called by Python when the class is called to create an object
- **Class attributes are shared by all instances, instance attributes belong to one object**
- **`__str__` and `__repr__` decide how an object is printed**
- **Attributes are public; a property adds a rule without changing the calling code :**
  - Start with a plain attribute, turn it into a property the day it needs a rule
- **A leading underscore marks an attribute as internal, by convention**
- **A class can extend another, and calls `super().__init__()` to set up the parent part**

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

- ▶ **Type Hinting**
- Python Libraries

- Type Hinting Recap
- Basic Type Syntax
- Typing Collections
- Optional Values
- Hints Are Not Enforced
- Checking with mypy
- Closed Sets with Literal

SpiralTrain developer courses

Type Hinting Recap

- **Annotations have appeared throughout the course, module by module :**
  - Variables in Variables and Data Types, `age: int = 25`
  - Collections in Data Structures, `list[int]` and `dict[str, int]`
  - Signatures in Functions, `def area(w: float, h: float) -> float`
  - Constructors and methods in Classes, `def __init__(self, name: str) -> None`
- **This appendix gathers them up and adds the three things left over :**
  - A value that may be absent, a closed set of literal values, and a checker
- **Python is dynamically typed and stays that way :**
  - A name refers to an object, and the object carries the type
  - Nothing forces a parameter to be the type the author had in mind
- **What an annotation buys is that the intent cannot drift from the code :**
  - Annotations can also go stale; Python does not enforce them at runtime
  - A checker such as mypy detects code that contradicts an annotation
  - The editor completes, renames and navigates with far more confidence
  - A checker reads the whole program and finds the mismatches

SpiralTrain developer courses

Basic Type Syntax

- To recap, a variable is annotated with a colon and the intended type :

```
age: int = 25
height: float = 1.87
name: str = "Alice"
is_student: bool = True
```

- A function states its parameters and its result :
  - Each parameter takes a colon, the result takes an arrow

```
def area(width: float, height: float) -> float:
    return width * height
```

- A function that returns nothing is annotated `None` :

```
def log_message(message: str) -> None:
    print(message)
```

demo01
annotations

SpiralTrain developer courses

Typing Collections

- And a container is written with the type of what it holds in brackets :

```
scores: list[int] = [10, 20, 30]
names: set[str] = {"Alice", "Bob"}
ages: dict[str, int] = {"Alice": 30, "Bob": 25}
```

- A tuple states the type of every position it has :

```
point: tuple[int, int] = (3, 4)
row: tuple[str, int, float] = ("Audi", 2019, 52643.0)
```

- A tuple of unknown length repeats one type with an ellipsis :

```
readings: tuple[float, ...] = (1.5, 2.5, 3.5, 4.5)
```

Optional Values

- A parameter that may be absent is typed with `| None` :

```
def greet(name: str | None = None) -> str:
    if name is None:
        return "Hello, stranger"
    return f"Hello, {name}"
```

- The default `None` is what makes the annotation necessary :
  - Writing `name: str = None` claims a type the default violates
- The same notation describes a value of either of two types :

```
def parse(value: str | int) -> int:
    return int(value)
```

demo03
optional

SpiralTrain developer courses

Hints Are Not Enforced

- **The interpreter ignores annotations while the program runs :**
  - No conversion happens, and no check happens
  - A wrong type is passed just as happily as a right one

```
def double(n: int) -> int:
    return n * 2


print(double("ab"))  # prints abab, no error is raised
```

- **Annotations are stored, and can be read at runtime :**

```
print(double.__annotations__)  # {'n': <class 'int'>, 'return': <class 'int'>}
```

- **So a hint is worth exactly as much as the tooling that reads it :**
  - Which is why the next slide runs a checker over it

demo04
not_enforced

SpiralTrain developer courses

Checking with mypy

- `mypy` reads the annotations and reports what does not agree :

```
uv add --dev mypy
uv run mypy 05_mypy_error.py
```

- Given a call that contradicts the signature :

```
def double(n: int) -> int:
    return n * 2


result: int = double("ab")
```

- The checker reports it without the program ever running :

```
error: Argument 1 to "double" has incompatible type "str"; expected "int"
Found 1 error in 1 file (checked 1 source file)
```

demo05
mypy_error

Closed Sets with Literal

- `typing.Literal` restricts a type to an exact set of allowed values :
  - A plain `str` accepts any text, whereas `Literal` rejects any value outside the declared options

```
from typing import Literal

type TrafficLight = Literal["red", "amber", "green"]


def change_light(color: TrafficLight) -> None:
    print(f"Traffic light is now {color}")


change_light("green")
change_light("blue")
```

- When checked with `mypy`, a value outside the set is flagged immediately :

```
06_literal.py:23: error: Argument 1 to "change_light" has incompatible type "Literal['blue']"; expected "Lit
Found 1 error in 1 file (checked 1 source file)
```

demo06
literal

SpiralTrain developer courses

Summary : Type Hinting

- **`uv` manages the interpreter, the environment and the packages :**
  - `uv python install` , `uv init` , `uv add` , `uv sync` , `uv run`
  - `pyproject.toml` states what is wanted, `uv.lock` pins what was resolved
- **`ruff` reports style and correctness problems without running the code :**
  - `ruff check .` and `ruff check . --fix` , configured in `ruff.toml`
- **Type hints record in the signature what a function expects and returns :**
  - `age: int` for a variable, `def f(a: int) -> str` for a function
  - `list[int]` , `dict[str, int]` and `tuple[int, int]` for containers
  - `str | None` for a value that may be absent
  - `Literal["red", "amber", "green"]` for a closed set of literal values
- **Annotations are ignored at runtime :**
  - Their value comes from the editor and from a checker such as `mypy`

exercise12
Type Hinting

SpiralTrain developer courses

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes

APPENDIX

- Type Hinting
- ▶ **Python Libraries**
  - Standard Library Overview
  - The datetime Module
  - Formatting and Parsing Dates
  - Date Arithmetic with timedelta
  - The json Module
  - Reading and Writing JSON Files
  - JSON and Python Types
  - Regular Expressions with re
  - Matching and Searching
  - Groups and Capturing
  - Substitution and Splitting
  - Command-Line Arguments with sys.argv
  - Testing with pytest
  - Reading a Test Failure
  - Exceptions and Parametrized Tests

SpiralTrain developer courses

Standard Library Overview

- **Python ships with a large standard library :**
  - Available on every installation, with nothing to add and nothing to pin
  - Reach for it before reaching for a third party package
- **The modules this appendix covers :**
  - `datetime` , dates, times and the arithmetic between them
  - `json` , reading and writing the interchange format of the web
  - `re` , regular expressions for searching and rewriting text
  - `sys.argv` , inspecting command-line arguments passed to a script
- **`pytest` is third-party software for testing with plain assertions :**
  - Exercise 13 installs it with `uv add --dev pytest`
- **Others worth knowing about :**
  - `pathlib` for filesystem paths, `csv` for tabular files
  - `collections` for counters, deques and default dictionaries
  - `itertools` for combinations, grouping and infinite sequences
  - `argparse` for flags, options and generated help messages
- **Full reference at `https://docs.python.org/3/library/`**

SpiralTrain developer courses

The datetime Module

- **Three types cover almost every need :**
  - `date` is a calendar day with no time on it
  - `time` is a time of day with no date on it
  - `datetime` is both together

```
from datetime import date, datetime, time

today = date(2026, 8, 15)
midday = time(12, 30, 0)
launch = datetime(2026, 8, 15, 12, 30, 0)

print(launch.year, launch.month, launch.day)
print(launch.weekday())  # Monday is 0, so this prints 5
```

- **The current moment comes from the class itself :**
  - `date.today()` and `datetime.now()`

demo01
datetime_basics

SpiralTrain developer courses

Formatting and Parsing Dates

- `strftime` formats a datetime as text, `strptime` reads text back :
  - The `f` is for format, the `p` is for parse

```
launch.strftime("%d-%m-%Y %H:%M")  # 15-08-2026 12:30
launch.strftime("%A %d %B %Y")     # Saturday 15 August 2026

datetime.strptime("24-12-2026 18:00", "%d-%m-%Y %H:%M")
```

- Common directives :
  - `%Y` four digit year, `%m` month, `%d` day
  - `%H` hour, `%M` minute, `%S` second
  - `%A` weekday name, `%B` month name
- ISO 8601 needs no format string in either direction :

```
launch.isoformat()  # 2026-08-15T12:30:00
datetime.fromisoformat("2026-12-24T18:00:00")
```

demo02
datetime_format

SpiralTrain developer courses

Date Arithmetic with timedelta

- Subtracting two dates gives a `timedelta`, a duration :

```
from datetime import date, timedelta

start = date(2026, 8, 15)
end = date(2026, 12, 24)
span = end - start
print(span.days)  # 131
```

- Adding a `timedelta` to a date gives another date :

```
start + timedelta(weeks=1)   # 2026-08-22
start + timedelta(days=100)  # 2026-11-23
start - timedelta(days=1)    # 2026-08-14
```

- A `timedelta` is built from any combination of units :
  - `days`, `hours`, `minutes`, `seconds`, `weeks`
  - It normalises itself, so 25 hours becomes one day and one hour

demo03
datetime_arithmetic

SpiralTrain developer courses

The json Module

- **JSON is the interchange format most web services speak :**
  - Text, language independent, and mapped closely onto Python types
- **`dumps` writes a string, `loads` reads one :**

```
import json

book = {"title": "Fluent Python", "pages": 1012, "sequel": None}
text = json.dumps(book, indent=2, sort_keys=True)
restored = json.loads(text)
```

- **`indent` and `sort_keys` are for people reading the output :**
  - Leave them out and the result is compact, which is what you send

```
{
  "pages": 1012,
  "sequel": null,
  "title": "Fluent Python"
}
```

demo04
json_strings

SpiralTrain developer courses

Reading and Writing JSON Files

- **`dump` and `load` take a file object rather than a string :**
  - The names are the same without the trailing `s`

```
import json
from pathlib import Path

path = Path("book.json")

with path.open("w", encoding="utf-8") as f:
    json.dump(book, f, indent=2)

with path.open(encoding="utf-8") as f:
    restored = json.load(f)
```

- **Always give an encoding :**
  - JSON is defined as UTF-8, and the platform default may not be
- **The four names are worth memorising :**
  - `dumps` and `loads` for strings, `dump` and `load` for files

demo05
json_files

SpiralTrain developer courses

JSON and Python Types

- The mapping between the two type systems is not quite one to one :

Python	JSON
`dict`	object
`list` , `tuple`	array
`str`	string
`int` , `float`	number
`True` / `False`	`true` / `false`
`None`	`null`

- Note what this costs on a round trip :
  - A `tuple` comes back as a `list`
  - Dictionary keys come back as strings, whatever they went in as
  - `set` , `datetime` and your own classes are not serialisable at all
- For those, convert to a supported type first, or pass `default=`

SpiralTrain developer courses

Regular Expressions with re

- **A regular expression is a pattern describing a set of strings :**
  - Written as a raw string, `r"..."`, so backslashes reach the engine intact
- **The character classes that do most of the work :**
  - `\d` digit, `\w` word character, `\s` whitespace
  - `.` any character, `[abc]` one of these, `[^abc]` none of these
- **The repetition operators :**
  - `*` zero or more, `+` one or more, `?` zero or one
  - `{3}` exactly three, `{2,4}` between two and four
- **Anchors tie a pattern to a position :**
  - `^` start of the string, `$` end of the string, `\b` word boundary

```
r"\d{4}-\d{2}-\d{2}"   # a date like 2026-08-15
r"^\w+@\w+\.\w{2,4}$"  # a very rough email address
```

SpiralTrain developer courses

Matching and Searching

- **Four functions cover most use, and they differ in where they look :**

```
import re

text = "Order 1234 shipped, order 5678 pending"

re.match(r"Order", text)     # only tries position zero
re.match(r"shipped", text)   # None, even though the word is present
re.search(r"shipped", text)  # scans the whole string, finds it
re.findall(r"\d+", text)     # ['1234', '5678']
print(re.fullmatch(r"[a-z]+", "hello") is not None)  # True; whole string
print(re.fullmatch(r"[a-z]+", "hello!") is not None)  # False
```

- **`match` , `fullmatch` and `search` return a match object, or `None` :**
  - So the result is tested before it is used
- **Flags change how the pattern is applied :**
  - `re.IGNORECASE` , `re.MULTILINE` , `re.DOTALL`

demo06
regex_search

SpiralTrain developer courses

Groups and Capturing

- Parentheses capture the part of the match you actually want :

```
line = "2026-08-15 12:30 ERROR disk full"
m = re.search(r"(\d{4})-(\d{2})-(\d{2}) (\d{2}:\d{2}) (\w+) (.*)", line)

m.group(0)  # the whole match
m.group(1)  # 2026, the first group
m.groups()  # ('2026', '08', '15', '12:30', 'ERROR', 'disk full')
```

- Named groups read better once there are more than two :

```
named = re.search(r"(?P<level>ERROR|WARN|INFO) (?P<message>.*)", line)
named.group("level")    # ERROR
named.group("message")  # disk full
```

demo07
regex_groups

SpiralTrain developer courses

Substitution and Splitting

- `sub` replaces every match in the string :

```
text = "Call 020-1234567 or 010-7654321 today"

re.sub(r"\d", "x", text)  # Call xxx-xxxxxxx or xxx-xxxxxxx today
re.sub(r"\d{3}-\d{7}", "[redacted]", text)
```

- A backreference in the replacement reuses a captured group :

```
re.sub(r"(\d{3})-(\d{7})", r"\2-\1", text)
# Call 1234567-020 or 7654321-010 today
```

- `re.split` accepts a pattern where `str.split` accepts only a literal :

```
re.split(r"\s+or\s+", text)
# ['Call 020-1234567', '010-7654321 today']
```

demo08
regex_sub

Command-Line Arguments with sys.argv

- `sys.argv` holds the arguments passed to a Python script as a list of strings :
  - `sys.argv[0]` is the name or path of the script itself
  - `sys.argv[1:]` contains any arguments passed after the script name

```
import sys

print(f"Script name : {sys.argv[0]}")
print(f"Arguments   : {sys.argv[1:]}")
```

- Every argument arrives as a string :
  - Convert numbers explicitly with `int()` or `float()`
  - Inspect `len(sys.argv)` to check how many arguments were supplied
- For option flags, defaults and help messages, `argparse` is the next step :
  - `argparse` handles `--flag` options, generates `--help` and parses types
  - `sys.argv` is the underlying list `argparse` reads from

demo09
sys_argv

SpiralTrain developer courses

Testing with pytest

- In `pytest`, a test is a function whose name starts with `test_`:
  - No test case class to subclass and no special assertion methods
  - Plain `assert` statements check the expected conditions
- `pytest` is installed in the project with `uv add --dev pytest`:

```python
def add(a: int, b: int) -> int:
    return a + b


def test_positive_numbers() -> None:
    assert add(2, 3) == 5


def test_with_zero() -> None:
    assert add(0, 7) == 7
```

- Run all discovered tests with `uv run pytest` (verbose with `-v`):

```
uv run pytest -v
```

demo10
pytest_basic

SpiralTrain developer courses

Reading a Test Failure

- When an `assert` fails, `pytest` explains exactly what went wrong :
  - It reconstructs intermediate expressions and shows where the values diverged
- Running `uv run pytest 11_pytest_failure.py` on a deliberately failing test :

```
_____________________________________ test_wrong_sum _____________________________________

    def test_wrong_sum() -> None:
>       assert add(2, 3) == 6
E       assert 5 == 6
E        +  where 5 = add(2, 3)

11_pytest_failure.py:15: AssertionError
=========================== short test summary info ============================
FAILED 11_pytest_failure.py::test_wrong_sum - assert 5 == 6
```

- The report shows the source line, the evaluated values, and the summary line :
  - No custom assertion methods like `assertEqual` are needed for clear output

demo11
pytest_failure

Exceptions and Parametrized Tests

- `pytest.raises` checks that a block of code raises an expected exception :
  - Used as a context manager; the test passes only if the exception occurs

```
with pytest.raises(ValueError):
    divide(10, 0)
```

- `@pytest.mark.parametrize` runs one test function across multiple inputs :
  - Each tuple in the list runs as an independent test case

```
@pytest.mark.parametrize(
    ("a", "b", "expected"),
    [
        (10, 2, 5.0),
        (9, 3, 3.0),
        (5, 2, 2.5),
    ],
)
def test_divide(a: int, b: int, expected: float) -> None:
    assert divide(a, b) == expected
```

demo12
pytest_features

SpiralTrain developer courses

Summary : Python Libraries

- **`datetime` covers dates, times and durations :**
  - `date` , `time` , `datetime` and `timedelta`
  - `strftime` formats and `strptime` parses using a pattern
  - `isoformat()` formats; `fromisoformat()` parses, without a format pattern
- **`json` converts between Python objects and JSON text :**
  - `dumps` and `loads` for strings, `dump` and `load` for files
  - A tuple returns as a list, and keys return as strings
- **`re` searches and rewrites text with patterns :**
  - `match` at the start, `search` anywhere, `findall` for every hit
  - Parentheses capture, and `sub` replaces using what was captured
- **`sys.argv` exposes command-line arguments as a list of strings :**
  - `argparse` is the next step for options, flags and automatic help
- **`pytest` runs checks over the code automatically :**
  - Test functions named `test_` and plain `assert` statements
  - `pytest.raises` for exceptions and `@pytest.mark.parametrize` for tables